

Sách Ôn Tập Lập Trình Thi Đấu Nâng Cao

SÁCH ÔN TẬP — TÀI LIỆU HỌC TẬP RÚT GỌN

Biên soạn bởi HackaGen
Ngày xuất bản: 06/09/2026

LỜI NÓI ĐẦU

Chào mừng bạn đến với thế giới đầy thử thách và hấp dẫn của lập trình thi đấu. Cuốn sách ôn tập này được biên soạn dành riêng cho những sinh viên và lập trình viên có đam mê với thuật toán, đặc biệt là những ai đang nuôi dưỡng ước mơ tham gia các kỳ thi danh giá như Olympic Tin học Quốc tế (IOI) hay Kỳ thi Lập trình Sinh viên Quốc tế (ICPC). Dù bạn là người mới bắt đầu tìm hiểu về thuật toán hay đã có kinh nghiệm lập trình cơ bản, tài liệu này sẽ là người bạn đồng hành đáng tin cậy trên hành trình của bạn.

Nội dung sách được cấu trúc một cách hệ thống, bắt đầu từ những viên gạch nền tảng như phân tích độ phức tạp, các kỹ thuật lập trình C++ hiệu quả, và kiến thức toán học cần thiết. Tiếp theo, chúng ta sẽ cùng nhau đi sâu vào các phương pháp thiết kế thuật toán kinh điển như tìm kiếm vét cạn, quy hoạch động, và các thuật toán trên đồ thị từ cơ bản đến nâng cao. Cuối cùng, sách sẽ giới thiệu các chủ đề chuyên sâu hơn như lý thuyết số, hình học tính toán và xác suất, những mảng kiến thức thường xuất hiện trong các cuộc thi cấp độ cao.

Để tận dụng tối đa cuốn sách này, chúng tôi khuyến khích bạn không chỉ đọc hiểu lý thuyết mà còn chủ động thực hành. Hãy thử tự mình cài đặt lại các thuật toán, giải quyết các bài tập liên quan trên các nền tảng như Codeforces hay Topcoder. Hãy nhớ rằng, trở thành một lập trình viên thi đấu giỏi là một quá trình đòi hỏi sự kiên trì và nỗ lực không ngừng. Cuốn sách này không phải là một bộ sưu tập các lời giải, mà là một tấm bản đồ giúp bạn rèn luyện tư duy giải quyết vấn đề một cách sáng tạo và hiệu quả. Chúc bạn có một hành trình học tập đầy hứng khởi và gặt hái được nhiều thành công!

TÓM TẮT NỘI DUNG

Cuốn sách này là một cẩm nang ôn tập chuyên sâu, dẫn dắt người đọc từ những khái niệm nền tảng trong lập trình thi đấu như độ phức tạp thời gian và kỹ thuật C++, đến các thuật toán cốt lõi về tìm kiếm, quy hoạch động, và lý thuyết đồ thị. Sách tiếp tục mở rộng sang các chủ đề nâng cao bao gồm lý thuyết số, hình học tính toán và xác suất. Đây là tài liệu tự học và ôn luyện toàn diện dành cho những ai muốn chinh phục các cuộc thi thuật toán như IOI và ICPC.

MỤC LỤC

Chương 1: Nhập môn Lập trình Thi đấu	5
Lập trình thi đấu là gì?	5
Tư duy Thuật toán và Kỹ năng Triển khai	6
Các cuộc thi tiêu biểu: IOI, ICPC và Các Sân chơi Trực tuyến	7
Tài liệu tham khảo và Lộ trình học tập	7
Chương 2: Các Kỹ thuật Lập trình và Toán học Nền tảng	10
Tối ưu hóa mã nguồn với typedef và macros	10
Những cạm bẫy thường gặp khi dùng macro	11
Công thức tính tổng cho cấp số cộng và cấp số nhân	11
Các công thức tổng lũy thừa cơ bản	12
Chương 3: Phân tích Độ phức tạp Thời gian	14
Ký hiệu Big O và Các Quy tắc Tính toán Cơ bản	14
Phân tích Độ phức tạp qua Vòng lặp và Đệ quy	15
Các Lớp Độ phức tạp Phổ biến	15
Thuật toán Đa thức và Giới thiệu về NP-khó	16
Ước tính Hiệu quả trong Thực tế	16
Chương 4: Các Kỹ thuật Tìm kiếm và Thao tác Bit	18
Tìm kiếm vét cạn: Sinh tất cả tập hợp con bằng đệ quy	18
Kỹ thuật thao tác bit trong C++	19
Các hàm _builtin để đếm bit	20
Mở rộng tìm kiếm nhị phân: Tìm giá trị lớn nhất	20
Chương 5: Quy hoạch động trên Tập hợp con	21
Bài toán Tổng trên các Tập hợp con	21
Ý tưởng Quy hoạch động và Công thức Truy hồi	22
Cách cài đặt hiệu quả với Mảng và Bitmask	23
Phân tích Độ phức tạp	24
Chương 6: Lý thuyết Đồ thị Cơ bản và Thuật toán Tìm chu trình	26
Các Thuật Ngữ Đồ Thị Cơ Bản	26
Tính Liên Thông và Thành Phần Liên Thông Mạnh	27
Cây: Một Dạng Đồ Thị Đặc Biệt	27
Thuật Toán Tìm Chu Trình Của Floyd: Con Rùa và Con Thỏ	28

Chương 7: Thành phần Liên thông mạnh và Truy vấn trên Cây	30
Đồ thị Liên thông mạnh và Thành phần Liên thông mạnh	30
Đồ thị Thành phần (Component Graph)	31
Thuật toán Kosaraju và Ứng dụng trong bài toán 2-SAT	31
Tìm Tổ tiên thứ k bằng Binary Lifting	33
Chương 8: Các Chủ đề Nâng cao: Số học, Hình học và Xác suất	35
Lý thuyết Số Ứng dụng	35
Xác suất và Thuật toán Ngẫu nhiên	36
Các Kỹ thuật trong Hình học Tính toán	37
Thuật toán Andrew tìm Bao lồi	38

Chương 1: Nhập môn Lập trình Thi đấu

Chào mừng bạn đến với thế giới của lập trình thi đấu, một lĩnh vực hấp dẫn nơi tư duy thuật toán và kỹ năng lập trình đỉnh cao hội tụ. Đây không chỉ là một môn thể thao trí tuệ mà còn là một hành trình rèn luyện khả năng phân tích và giải quyết vấn đề một cách sáng tạo và hiệu quả. Chương này sẽ đặt những viên gạch đầu tiên, giới thiệu bản chất của lập trình thi đấu, phác thảo các kỹ năng cần thiết, và điểm qua những đấu trường danh giá như IOI và ICPC. Việc nắm vững những kiến thức nền tảng này là bước khởi đầu không thể thiếu cho bất kỳ ai muốn chinh phục những thử thách thuật toán phức tạp.

Mục tiêu học tập

- Giải thích được bản chất của lập trình thi đấu và hai thành phần cốt lõi của nó: thiết kế thuật toán và triển khai thuật toán.
- Phân biệt được đặc điểm, thể thức và đối tượng tham gia của các cuộc thi lập trình phổ biến như IOI và ICPC.
- Nhận biết được các ngôn ngữ lập trình, công cụ và tài liệu tham khảo hữu ích cho người mới bắt đầu.
- Hiểu được tầm quan trọng của việc rèn luyện song song cả tư duy lý thuyết và kỹ năng lập trình thực tế.

Lập trình thi đấu là gì?

Lập trình thi đấu (Competitive Programming) là một môn thể thao trí tuệ kết hợp hai lĩnh vực chính: thiết kế thuật toán và triển khai thuật toán. Về bản chất, bạn sẽ được cho một bài toán với các yêu cầu và ràng buộc cụ thể, và nhiệm vụ của bạn là viết một chương trình máy tính có thể giải quyết bài toán đó trong giới hạn thời gian và bộ nhớ cho phép.

Thành phần đầu tiên, thiết kế thuật toán, đòi hỏi tư duy toán học và khả năng giải quyết vấn đề một cách sáng tạo. Trọng tâm của nó là nghĩ ra một phương pháp, một thuật toán, không chỉ đúng mà còn phải hiệu quả. Một thuật toán đúng nhưng quá chậm sẽ không được chấp nhận. Do đó, cốt lõi của việc thiết kế thường xoay quanh việc phát minh ra một thuật toán đủ nhanh để xử lý các bộ dữ liệu lớn. Kiến thức lý thuyết về thuật toán là cực kỳ quan trọng, vì các giải pháp thường là sự kết hợp giữa các kỹ thuật đã biết và những ý tưởng mới mẻ, độc đáo. Những kỹ thuật này cũng chính là nền tảng cho nghiên cứu khoa học về thuật toán.

Thành phần thứ hai, triển khai thuật toán, yêu cầu kỹ năng lập trình vững chắc. Trong các cuộc thi, giải pháp của bạn sẽ được chấm điểm tự động bằng cách chạy chương trình của bạn với một bộ dữ liệu thử (test cases). Điều này có nghĩa là, ý tưởng thuật toán đúng thôi là chưa đủ; việc triển khai thành mã nguồn cũng phải hoàn toàn chính xác. Một sai sót nhỏ trong lập trình cũng có thể dẫn đến kết quả sai. Có thể hình dung việc thiết kế thuật toán giống như một kiến trúc sư vẽ ra bản thiết

kế chi tiết cho một tòa nhà – phải đảm bảo tính thẩm mỹ, công năng và sự vững chắc. Còn việc triển khai chính là quá trình thi công của các kỹ sư và công nhân – họ phải tuân thủ chính xác bản vẽ, vì chỉ một sai lệch nhỏ cũng có thể ảnh hưởng đến toàn bộ công trình.

Tư duy Thuật toán và Kỹ năng Triển khai

Để thành công trong lập trình thi đấu, việc rèn luyện song song cả tư duy thuật toán và kỹ năng triển khai là tối quan trọng. Tư duy thuật toán là khả năng phân tích một vấn đề, nhận ra cấu trúc ẩn sau nó và áp dụng hoặc sáng tạo ra một phương pháp hiệu quả để giải quyết. Nó không chỉ dừng lại ở việc tìm ra một lời giải, mà còn là tìm ra lời giải tốt nhất có thể trong các ràng buộc cho trước, đặc biệt là về thời gian chạy, thường được phân tích qua khái niệm độ phức tạp thời gian (ví dụ: $O(n)$, $O(n \log n)$).

Kỹ năng triển khai, mặt khác, là khả năng biến những ý tưởng thuật toán thành một chương trình máy tính chạy được và không có lỗi. Trong môi trường thi đấu, phong cách lập trình được ưa chuộng là súc tích và đi thẳng vào vấn đề. Các thí sinh phải viết mã rất nhanh vì thời gian có hạn. Không giống như trong ngành kỹ thuật phần mềm truyền thống, các chương trình trong thi đấu thường khá ngắn (thường không quá vài trăm dòng mã) và không cần phải bảo trì sau cuộc thi. Do đó, sự rõ ràng, tối giản và tốc độ viết mã được ưu tiên hàng đầu.

Hiện tại, các ngôn ngữ lập trình phổ biến nhất trong các cuộc thi là C++, Python và Java. Theo thống kê tại Google Code Jam 2017, trong số 3000 thí sinh hàng đầu, 79% sử dụng C++, 16% sử dụng Python và 8% sử dụng Java. Nhiều người cho rằng C++ là lựa chọn tốt nhất vì đây là một ngôn ngữ rất hiệu quả và thư viện chuẩn (Standard Library) của nó chứa một bộ sưu tập lớn các cấu trúc dữ liệu và thuật toán có sẵn. Tuy nhiên, việc thành thạo nhiều ngôn ngữ cũng là một lợi thế. Ví dụ, Python có thể là lựa chọn tốt khi bài toán yêu cầu xử lý số nguyên lớn, vì nó tích hợp sẵn các phép toán cho loại dữ liệu này. Dù vậy, hầu hết các bài toán đều được thiết kế để việc sử dụng một ngôn ngữ cụ thể không tạo ra lợi thế không công bằng.

Đối với C++, một đoạn mã mẫu thường dùng trong thi đấu có dạng:

```
include <bits/stdc++.h>
using namespace std;
int main()
// Tối ưu hóa nhập xuất
ios::syncwith_stdio(0);
cin.tie(0);
// Phần giải thuật toán
`
```

Dòng `#include <bits/stdc++.h>` là một tính năng của trình biên dịch g++ cho phép bao gồm toàn bộ thư viện chuẩn, giúp bạn không cần phải include riêng lẻ từng thư viện như `iostream` hay `vector`. Để tăng tốc độ nhập xuất, hai dòng `ios::syncwith_stdio(0);` và `cin.tie(0);` thường được thêm vào đầu hàm `main`. Ngoài ra,

việc sử dụng "" để xuống dòng sẽ nhanh hơn endl vì endl` luôn thực hiện một thao tác xả bộ đệm (flush).

Các cuộc thi tiêu biểu: IOI, ICPC và Các Sân chơi Trực tuyến

Hai trong số những cuộc thi lập trình danh giá và lâu đời nhất là Kỳ thi Olympic Tin học Quốc tế (IOI) và Kỳ thi Lập trình Đồng đội Quốc tế ICPC.

IOI (International Olympiad in Informatics) là một cuộc thi thường niên dành cho học sinh trung học phổ thông. Mỗi quốc gia được cử một đội gồm tối đa bốn học sinh tham dự. Mặc dù tham gia theo đội tuyển quốc gia, các thí sinh thi đấu với tư cách cá nhân. Kỳ thi bao gồm hai ngày thi, mỗi ngày kéo dài năm giờ. Trong mỗi ngày thi, thí sinh được yêu cầu giải quyết ba bài toán thuật toán với độ khó khác nhau. Các bài toán thường được chia thành các bài toán con (subtasks), mỗi subtask mang lại một số điểm nhất định, cho phép thí sinh có thể nhận điểm từng phần ngay cả khi chưa có lời giải hoàn hảo. Việc tuyển chọn thí sinh tham dự IOI được thực hiện thông qua các kỳ thi quốc gia.

ICPC (International Collegiate Programming Contest) là cuộc thi lập trình thường niên dành cho sinh viên đại học. Không giống như IOI, ICPC là một cuộc thi đồng đội. Mỗi đội gồm ba sinh viên, cùng nhau làm việc trên một máy tính duy nhất để giải quyết một bộ đề. Các đội có năm giờ để giải quyết khoảng mười bài toán thuật toán. Một lời giải chỉ được chấp nhận khi nó vượt qua tất cả các bộ dữ liệu thử một cách hiệu quả. ICPC có cấu trúc nhiều vòng, từ cấp khu vực đến vòng chung kết toàn cầu (World Finals). Một điểm đặc trưng của ICPC là bảng điểm sẽ bị "đóng băng" trong giờ thi đấu cuối cùng, khiến các đội không thể thấy kết quả nộp bài của đối thủ, tạo nên sự kịch tính cho đến phút chót. Nhìn chung, ICPC đòi hỏi kiến thức rộng hơn IOI, đặc biệt là về các kỹ năng toán học.

Bên cạnh IOI và ICPC, có rất nhiều cuộc thi và sân chơi trực tuyến mở cho tất cả mọi người. Hiện tại, trang web năng động nhất là Codeforces, nơi tổ chức các kỳ thi gần như hàng tuần và phân loại người tham gia thành hai hạng (Div1 cho người có kinh nghiệm và Div2 cho người mới bắt đầu). Các nền tảng khác bao gồm AtCoder, CS Academy, HackerRank và Topcoder. Ngoài ra, một số công ty công nghệ lớn cũng tổ chức các cuộc thi trực tuyến với vòng chung kết tại chỗ như Facebook Hacker Cup, Google Code Jam và Yandex.Algorithm. Đây cũng là một cách hiệu quả để các công ty tuyển dụng nhân tài và để các lập trình viên chứng tỏ kỹ năng của mình.

Tài liệu tham khảo và Lộ trình học tập

Trở thành một lập trình viên thi đấu giỏi là một quá trình dài hơi, nhưng đó cũng là một cơ hội để học hỏi rất nhiều điều. Bằng cách dành thời gian đọc sách, giải bài tập và tham gia các cuộc thi, bạn chắc chắn sẽ có được một sự hiểu biết sâu sắc và toàn diện về thuật toán. Lộ trình học tập không phải là một đường thẳng, mà là một vòng lặp liên tục của việc học lý thuyết, thực hành và thi đấu để kiểm tra kỹ năng.

Để bắt đầu, có một số cuốn sách được đánh giá cao dành riêng cho lập trình thi đấu. Dành cho người mới, hai cuốn sách phổ biến là "Programming Challenges: The Programming Contest Training Manual" của S. S. Skiena và M. A. Revilla, và "Competitive Programming 3: The New Lower Bound of Programming Contests" của S. Halim và F. Halim. Đối với những người tìm kiếm thử thách nâng cao, cuốn "Looking for a Challenge? The Ultimate Problem Set from the University of Warsaw Programming Competitions" của K. Diks và cộng sự cung cấp những tài liệu chuyên sâu. Cuốn sách này bạn đang đọc cũng được thiết kế đặc biệt cho những sinh viên muốn học thuật toán để tham gia IOI hoặc ICPC.

Bên cạnh đó, các sách giáo khoa kinh điển về thuật toán cũng là nguồn tài liệu vô giá. Một số đầu sách nổi tiếng bao gồm "Introduction to Algorithms" (thường được gọi là CLRS) của Cormen, Leiserson, Rivest và Stein; "Algorithm Design" của J. Kleinberg và É. Tardos; và "The Algorithm Design Manual" của S. S. Skiena. Những cuốn sách này cung cấp nền tảng lý thuyết vững chắc cho hầu hết các chủ đề bạn sẽ gặp trong thi đấu.

Lý thuyết phải luôn đi đôi với thực hành. Các nền tảng luyện tập trực tuyến là nơi tốt nhất để áp dụng kiến thức. Một số quốc gia tổ chức các kỳ thi thực hành mở, chẳng hạn như Croatian Open Competition in Informatics và USA Computing Olympiad (USACO), đây là những nguồn bài tập tuyệt vời cho các thí sinh tiềm năng của IOI. Việc thường xuyên giải bài trên các hệ thống này và tham gia các cuộc thi trực tuyến trên Codeforces hay AtCoder là cách hiệu quả nhất để mài giũa kỹ năng và tích lũy kinh nghiệm thực chiến.

Điểm cốt lõi

- > Lập trình thi đấu là sự kết hợp giữa hai kỹ năng cốt lõi: thiết kế thuật toán (đúng và hiệu quả) và triển khai thuật toán (lập trình chính xác).
- > C++ là ngôn ngữ được ưa chuộng nhất nhờ hiệu suất cao và thư viện chuẩn mạnh mẽ, nhưng việc biết các ngôn ngữ khác như Python cũng có lợi thế trong một số trường hợp cụ thể.
- > IOI là cuộc thi cá nhân dành cho học sinh phổ thông, chấm điểm theo subtask, trong khi ICPC là cuộc thi đồng đội dành cho sinh viên đại học, yêu cầu giải pháp phải đúng hoàn toàn.
- > Học tập là một quá trình liên tục bao gồm việc nghiên cứu lý thuyết từ sách, thực hành giải bài trên các hệ thống chấm điểm trực tuyến, và tham gia các cuộc thi để cọ xát.
- > Phong cách lập trình trong thi đấu ưu tiên sự súc tích, rõ ràng và tốc độ, khác với các yêu cầu về bảo trì trong kỹ thuật phần mềm truyền thống.

Câu hỏi ôn tập

1. Hai trụ cột chính của lập trình thi đấu là gì và tại sao cả hai đều quan trọng như nhau để thành công?

2. So sánh và chỉ ra những điểm khác biệt cơ bản về hình thức thi đấu, đối tượng tham gia và cách tính điểm giữa IOI và ICPC.
3. Tại sao C++ thường được coi là lựa chọn hàng đầu trong lập trình thi đấu? Trong trường hợp nào một ngôn ngữ như Python có thể là lựa chọn tốt hơn?
4. Ngoài việc tham gia các kỳ thi chính thức, bạn có thể làm gì để rèn luyện và nâng cao kỹ năng lập trình thi đấu của mình một cách thường xuyên?

Chương 2: Các Kỹ thuật Lập trình và Toán học Nền tảng

Trong lập trình thi đấu, tốc độ không chỉ đến từ thuật toán hiệu quả mà còn từ khả năng viết mã nhanh chóng và chính xác. Chương này sẽ trang bị cho bạn những công cụ thiết yếu để đạt được điều đó. Chúng ta sẽ khám phá các kỹ thuật trong C++ như typedef và macros để rút ngắn mã nguồn, giúp bạn tập trung vào logic giải quyết vấn đề. Đồng thời, chương sách sẽ củng cố nền tảng toán học của bạn với các công thức tính tổng thường gặp, cho phép bạn thay thế những vòng lặp tốn kém bằng các phép tính tức thời. Việc nắm vững các kỹ thuật này là một bước đệm quan trọng, giúp bạn viết mã vừa ngắn gọn, vừa hiệu quả để chinh phục các bài toán phức tạp hơn.

Mục tiêu học tập

- Sử dụng typedef và macros để viết mã C++ ngắn gọn và dễ đọc hơn.
- Nhận biết và phòng tránh các lỗi tiềm ẩn nguy hiểm khi sử dụng macros có tham số.
- Áp dụng công thức tính tổng cấp số cộng và các công thức tổng lũy thừa cơ bản để giải quyết bài toán một cách hiệu quả.
- Phân biệt được khi nào nên sử dụng công thức toán học thay cho các giải pháp lặp.

Tối ưu hóa mã nguồn với typedef và macros

Viết mã ngắn gọn là một kỹ năng quan trọng trong lập trình thi đấu, bởi lẽ mỗi giây tiết kiệm được trong quá trình viết mã đều vô cùng quý giá. Vì lý do này, các lập trình viên thi đấu thường tìm cách định nghĩa các tên gọi ngắn hơn cho các kiểu dữ liệu và các đoạn mã lặp đi lặp lại. Hai công cụ phổ biến nhất để thực hiện điều này trong C++ là từ khóa typedef và cơ chế tiền xử lý macros.

typedef là một từ khóa cho phép bạn tạo một tên gọi khác, thường là ngắn hơn, cho một kiểu dữ liệu đã có. Ví dụ, kiểu dữ liệu long long rất hay được sử dụng nhưng lại khá dài để gõ. Bằng cách sử dụng typedef long long ll;, bạn có thể thay thế hoàn toàn long long bằng ll trong toàn bộ mã nguồn của mình. Điều này không chỉ giúp tiết kiệm thời gian gõ phím mà còn làm cho mã nguồn trông gọn gàng hơn, đặc biệt khi bạn cần khai báo nhiều biến thuộc kiểu này. Ví dụ, long long a = 123456789; có thể được viết lại thành ll a = 123456789;.

Sức mạnh của typedef không chỉ dừng lại ở các kiểu dữ liệu cơ bản. Nó đặc biệt hữu ích khi làm việc với các cấu trúc dữ liệu phức tạp từ Thư viện Chuẩn (STL). Chẳng hạn, thay vì phải gõ vector<int> hay pair<int, int> nhiều lần, bạn có thể định nghĩa typedef vector<int> vi; và typedef pair<int, int> pi;. Từ đó, việc khai báo một vector số nguyên hay một cặp số nguyên trở nên đơn giản và dễ đọc hơn rất nhiều, giúp mã nguồn của bạn mạch lạc và tập trung vào logic chính của thuật toán.

Một phương pháp khác để rút ngắn mã là sử dụng macros, được định nghĩa bằng từ khóa `#define`. Khác với `typedef`, macros hoạt động ở giai đoạn tiền xử lý, tức là nó sẽ tìm và thay thế một chuỗi ký tự này bằng một chuỗi ký tự khác trước khi mã được biên dịch. Ví dụ, bạn có thể định nghĩa `#define PB pushback` để thay thế cho lời gọi hàm `v.pushback(...)` bằng `v.PB(...)`. Tương tự, các macro như `#define F first` và `#define S second` rất phổ biến để truy cập các thành phần của một pair. Macro cũng có thể nhận tham số, cho phép rút gọn cả những cấu trúc phức tạp như vòng lặp. Ví dụ, với `#define REP(i,a,b) for (int i = a; i <= b; i++)`, một vòng lặp for tiêu chuẩn có thể được viết ngắn gọn thành `REP(i,1,n) ...`.

Những cạm bẫy thường gặp khi dùng macro

Mặc dù macros là một công cụ tiện lợi để rút ngắn mã, chúng lại tiềm ẩn những lỗi rất tinh vi và khó phát hiện. Nguyên nhân sâu xa nằm ở bản chất của chúng: macros chỉ đơn thuần là một cơ chế thay thế văn bản (text substitution) của bộ tiền xử lý, chứ không phải là một lời gọi hàm thực sự. Trình biên dịch không hề biết đến sự tồn tại của macro, nó chỉ làm việc với đoạn mã đã được thay thế. Điều này có thể dẫn đến các kết quả không mong muốn do thứ tự ưu tiên của các toán tử.

Hãy xem xét một ví dụ kinh điển. Giả sử bạn định nghĩa một macro để tính bình phương của một số: `#define SQ(a) aa`. Thoạt nhìn, macro này có vẻ đúng. Nếu bạn gọi `SQ(5)`, nó sẽ được thay thế thành `55` và cho ra kết quả `25`. Tuy nhiên, vấn đề sẽ xuất hiện khi tham số là một biểu thức. Nếu bạn gọi `cout << SQ(3+3);`, bộ tiền xử lý sẽ thay thế nó thành `cout << 3+3*3+3;`. Do phép nhân có độ ưu tiên cao hơn phép cộng, biểu thức này sẽ được tính là `3 + 9 + 3`, cho ra kết quả là `15`, thay vì $(3+3)^2 = 36$ như chúng ta kỳ vọng.

Để khắc phục triệt để vấn đề này, quy tắc vàng khi viết macro có tham số là: luôn luôn bao bọc mỗi tham số và toàn bộ biểu thức của macro trong một cặp dấu ngoặc đơn. Phiên bản đúng của macro tính bình phương sẽ là `#define SQ(a) ((a)(a))`. Bây giờ, khi gọi `SQ(3+3)`, nó sẽ được thay thế thành `((3+3)(3+3))`. Các cặp dấu ngoặc đơn này đảm bảo rằng biểu thức `3+3` được tính toán trước, sau đó kết quả `6` mới được nhân với chính nó, cho ra kết quả đúng là `36`. Việc tuân thủ quy tắc này sẽ giúp bạn tránh được những lỗi logic cực kỳ khó gỡ.

Để dễ hình dung, hãy tưởng tượng macro như một trợ lý chỉ biết làm theo lệnh một cách máy móc. Nếu bạn đưa cho trợ lý một công thức

Công thức tính tổng cho cấp số cộng và cấp số nhân

Toán học đóng một vai trò không thể thiếu trong lập trình thi đấu. Việc nắm vững các công thức toán học cơ bản cho phép chúng ta tìm ra các lời giải có dạng đóng (closed-form), giúp tính toán kết quả ngay lập tức thay vì phải thực hiện các vòng lặp tốn nhiều thời gian. Hai trong số các dạng dãy số cơ bản nhất là cấp số cộng và cấp số nhân.

Một cấp số cộng (arithmetic progression) là một dãy số trong đó hiệu số giữa hai phần tử liên tiếp bất kỳ là một hằng số, gọi là công sai. Ví dụ, dãy số 3, 7, 11, 15 là một cấp số cộng với công sai là 4. Thay vì dùng vòng lặp để cộng từng số lại với nhau, chúng ta có thể sử dụng một công thức đơn giản để tính tổng của nó. Công thức này đặc biệt hữu ích khi dãy số có hàng triệu phần tử.

Công thức tính tổng của một cấp số cộng gồm n số hạng, với số hạng đầu là a và số hạng cuối là b , là $S_n = \frac{n(a+b)}{2}$. Ý tưởng đằng sau công thức này rất trực quan: giá trị trung bình của mỗi số hạng trong dãy là $(a+b)/2$, và vì có n số hạng, tổng của chúng chính là số lượng nhân với giá trị trung bình. Áp dụng vào ví dụ trên, tổng $3+7+11+15$ có 4 số hạng, số đầu là 3, số cuối là 15. Tổng sẽ là $4 \times (3+15)/2 = 36$. Công thức này cho phép tính toán trong thời gian hằng số $O(1)$, hiệu quả vượt trội so với vòng lặp $O(n)$.

Một dạng dãy số quan trọng khác là cấp số nhân (geometric progression). Đây là một dãy số mà tỉ số giữa hai phần tử liên tiếp bất kỳ là một hằng số, gọi là công bội. Ví dụ, dãy 3, 6, 12, 24 là một cấp số nhân với công bội là 2. Tương tự như cấp số cộng, cấp số nhân cũng có công thức tính tổng của nó. Tuy nhiên, tài liệu này chỉ giới thiệu định nghĩa của cấp số nhân mà không đi sâu vào công thức tính tổng chi tiết.

Các công thức tổng lũy thừa cơ bản

Bên cạnh các cấp số, việc tính tổng của các lũy thừa bậc nguyên của các số tự nhiên đầu tiên cũng là một yêu cầu thường gặp trong các bài toán. Một tổng có dạng $\sum_{k=1}^n k^k = 1^k + 2^k + 3^k + \dots + n^k$, trong đó k là một số nguyên dương, luôn có một công thức dạng đóng là một đa thức bậc $k+1$ theo biến n . Việc biết và áp dụng các công thức này là một lợi thế lớn.

Trường hợp đơn giản và phổ biến nhất là khi $k=1$. Chúng ta có tổng của n số tự nhiên đầu tiên: $\sum_{k=1}^n k = 1 + 2 + 3 + \dots + n = n(n+1)/2$. Dễ dàng nhận thấy đây chính là một cấp số cộng với số hạng đầu $a=1$, số hạng cuối $b=n$, và có n số hạng. Áp dụng công thức cấp số cộng, ta cũng thu được kết quả tương tự. Công thức này cực kỳ hữu ích và xuất hiện trong rất nhiều bài toán từ cơ bản đến nâng cao.

Khi $k=2$, chúng ta cần tính tổng bình phương của n số tự nhiên đầu tiên. Công thức cho trường hợp này là: $\sum_{k=1}^n k^2 = 1^2 + 2^2 + 3^2 + \dots + n^2 = n(n+1)(2n+1)/6$. Việc ghi nhớ công thức này sẽ giúp bạn giải quyết các bài toán liên quan trong thời gian $O(1)$, trong khi một vòng lặp đơn giản sẽ mất thời gian $O(n)$. Sự khác biệt về hiệu suất sẽ trở nên cực kỳ rõ rệt khi n có giá trị lớn, ví dụ như 10^9 hoặc 10^{12} .

Trên thực tế, có một công thức tổng quát cho các tổng lũy thừa này, được gọi là công thức Faulhaber, nhưng nó khá phức tạp và thường không cần thiết phải ghi nhớ trong các kỳ thi lập trình. Đối với hầu hết các bài toán, việc nắm vững các công thức cho $k=1$ và $k=2$ đã là đủ. Điều quan trọng là nhận ra khi nào một bài toán có thể được đơn giản hóa bằng cách áp dụng các công thức toán học này.

Điểm cốt lõi

- > typedef và macros là hai công cụ mạnh mẽ trong C++ để rút ngắn mã và tăng tốc độ lập trình, nhưng cần sử dụng một cách có hiểu biết.
- > Luôn bao bọc các tham số và toàn bộ biểu thức của macro trong dấu ngoặc đơn để tránh các lỗi nghiêm trọng về thứ tự ưu tiên của toán tử.
- > Các công thức tính tổng (cấp số cộng, tổng lũy thừa) cho phép tính toán kết quả trong thời gian hằng số $O(1)$, hiệu quả hơn nhiều so với việc lặp qua từng phần tử.
- > Nhận dạng được cấu trúc toán học ẩn sau yêu cầu bài toán là chìa khóa để tìm ra lời giải tối ưu.
- > Viết mã ngắn gọn không chỉ để tiết kiệm thời gian gõ phím mà còn để làm cho logic chính của thuật toán trở nên rõ ràng và dễ gỡ lỗi hơn.

Câu hỏi ôn tập

1. Sự khác biệt cơ bản trong cách trình biên dịch xử lý một macro và một hàm inline là gì? Điều này dẫn đến những ưu và nhược điểm nào cho mỗi cách tiếp cận?
2. Cho macro `#define PRODUCT(x, y) x*y`. Biểu thức `PRODUCT(2+3, 4+5)` sẽ cho ra kết quả là bao nhiêu? Hãy giải thích và viết lại macro cho đúng.
3. Làm thế nào bạn có thể tính tổng của 50 số chẵn đầu tiên (2, 4, 6, ...) bằng cách sử dụng công thức tính tổng cấp số cộng mà không cần dùng vòng lặp?
4. Một bài toán yêu cầu tính $1^2 + 2^2 + \dots + N^2$ với N lên tới 10^{12} . Tại sao giải pháp dùng vòng lặp for là không khả thi và giải pháp nào sẽ được sử dụng thay thế?

Chương 3: Phân tích Độ phức tạp Thời gian

Trong lập trình thi đấu, việc tìm ra một thuật toán giải được bài toán là chưa đủ; thuật toán đó còn phải đủ nhanh. Hiệu quả của thuật toán là yếu tố then chốt quyết định bạn có giành được điểm tối đa hay không. Chương này giới thiệu về độ phức tạp thời gian, một công cụ toán học cho phép chúng ta ước tính thời gian chạy của một thuật toán dựa trên kích thước đầu vào. Bằng cách phân tích độ phức tạp, bạn có thể đánh giá một ý tưởng giải quyết có khả thi hay không ngay cả trước khi viết dòng mã đầu tiên, từ đó tiết kiệm thời gian và công sức đáng kể.

Mục tiêu học tập

- Giải thích được khái niệm độ phức tạp thời gian và ý nghĩa của ký hiệu Big O.
- Áp dụng được các quy tắc cơ bản để tính toán độ phức tạp cho các đoạn mã chứa vòng lặp, các pha liên tiếp và lời gọi đệ quy.
- Phân biệt được các lớp độ phức tạp phổ biến và nhận diện được dạng thuật toán tương ứng với chúng.
- Ước tính được độ phức tạp thời gian cần thiết cho một bài toán dựa trên giới hạn kích thước đầu vào.

Ký hiệu Big O và Các Quy tắc Tính toán Cơ bản

Độ phức tạp thời gian của một thuật toán được biểu diễn bằng ký hiệu Big O, viết là $O(\dots)$, trong đó phần ba chấm là một hàm toán học. Thông thường, biến n được dùng để chỉ kích thước của đầu vào. Ví dụ, nếu đầu vào là một mảng các số, n sẽ là kích thước của mảng; nếu đầu vào là một chuỗi, n sẽ là độ dài của chuỗi. Ký hiệu Big O không cho chúng ta biết chính xác số lượng phép tính mà thuật toán sẽ thực hiện, mà nó chỉ ra bậc tăng trưởng (order of magnitude) của thời gian chạy khi kích thước đầu vào tăng lên.

Một trong những quy tắc quan trọng nhất của phân tích Big O là bỏ qua các hằng số nhân. Điều này có nghĩa là một vòng lặp chạy $3n$ lần, $n+5$ lần, hay $n/2$ lần đều có độ phức tạp là $O(n)$. Lý do là khi n trở nên rất lớn, sự khác biệt giữa n và $5n$ không quan trọng bằng sự khác biệt giữa n và n^2 . Chúng ta chỉ quan tâm đến thuật ngữ có ảnh hưởng lớn nhất đến thời gian chạy. Tương tự, nếu một thuật toán bao gồm nhiều pha thực hiện nối tiếp nhau, độ phức tạp tổng thể sẽ là độ phức tạp của pha chậm nhất. Ví dụ, một chương trình có ba pha với độ phức tạp lần lượt là $O(n)$, $O(n^2)$, và $O(n)$ sẽ có độ phức tạp tổng thể là $O(n^2)$, vì pha $O(n^2)$ chính là "nút thắt cổ chai" của toàn bộ thuật toán.

Đôi khi, độ phức tạp của một thuật toán không chỉ phụ thuộc vào một yếu tố duy nhất. Trong những trường hợp như vậy, công thức Big O sẽ chứa nhiều biến. Ví dụ, nếu bạn có một vòng lặp lồng nhau, trong đó vòng lặp ngoài chạy n lần và vòng lặp trong chạy m lần, thì độ phức tạp của đoạn mã đó sẽ là $O(nm)$. Điều này thường xảy ra khi xử lý các cấu trúc dữ liệu hai chiều như ma trận hoặc khi so khớp hai tập dữ

liệu có kích thước khác nhau.

Phân tích Độ phức tạp qua Vòng lặp và Đệ quy

Nguyên nhân phổ biến nhất khiến một thuật toán trở nên chậm là do nó chứa các vòng lặp, đặc biệt là các vòng lặp lồng nhau. Một quy tắc chung là nếu có k vòng lặp lồng nhau, mỗi vòng lặp chạy qua đầu vào có kích thước n , thì độ phức tạp thời gian sẽ là $O(n^k)$. Ví dụ, một vòng lặp đơn duyệt qua một mảng n phần tử có độ phức tạp là $O(n)$. Nếu chúng ta lồng thêm một vòng lặp nữa bên trong để duyệt qua tất cả các cặp phần tử, độ phức tạp sẽ trở thành $O(n^2)$.

Đệ quy là một nguồn gây ra độ phức tạp khác cần được phân tích cẩn thận. Độ phức tạp của một hàm đệ quy được tính bằng cách nhân số lần hàm được gọi với độ phức tạp của một lần gọi. Xét một hàm $f(n)$ đơn giản, trong mỗi lần gọi nó chỉ thực hiện một vài phép tính cơ bản (độ phức tạp $O(1)$) và sau đó gọi $f(n-1)$. Khi bắt đầu với $f(n)$, hàm sẽ được gọi tổng cộng n lần (cho đến khi đạt đến trường hợp cơ sở), do đó tổng độ phức tạp là $n \times O(1) = O(n)$.

Tuy nhiên, mọi chuyện có thể trở nên phức tạp hơn rất nhiều. Hãy tưởng tượng một hàm $g(n)$ mà trong mỗi lần gọi, nó lại tự gọi chính nó hai lần: $g(n-1)$ và $g(n-1)$. Một lời gọi $g(n)$ sẽ tạo ra hai lời gọi $g(n-1)$, bốn lời gọi $g(n-2)$, và cứ thế tiếp tục. Tổng số lời gọi sẽ là một chuỗi cấp số nhân: $1 + 2 + 4 + \dots + 2^{n-1} = 2^n - 1$. Do đó, độ phức tạp của hàm này là $O(2^n)$. Điều này cho thấy các thuật toán đệ quy phân nhánh có thể nhanh chóng dẫn đến độ phức tạp theo hàm mũ, khiến chúng không khả thi cho các giá trị n lớn.

Các Lớp Độ phức tạp Phổ biến

Các thuật toán có thể được phân loại vào các lớp độ phức tạp khác nhau, cho biết tốc độ tăng thời gian chạy của chúng. Lớp $O(1)$ (hằng số) là nhanh nhất, thời gian chạy không phụ thuộc vào kích thước đầu vào; ví dụ điển hình là một công thức tính toán trực tiếp. Tiếp theo là $O(\log n)$ (logarit), thường xuất hiện trong các thuật toán chia để trị, nơi kích thước bài toán được giảm đi một nửa ở mỗi bước. Hãy tưởng tượng bạn đang tìm một từ trong từ điển bằng cách luôn mở ra trang ở giữa; số lần bạn cần lật trang tăng rất chậm so với tổng số trang, đó chính là bản chất của độ phức tạp logarit.

Lớp $O(\sqrt{n})$ (căn bậc hai) chậm hơn $O(\log n)$ nhưng nhanh hơn $O(n)$. Các thuật toán này đôi khi chia dữ liệu thành $\sqrt{n}$ khối, mỗi khối có $\sqrt{n}$ phần tử để cân bằng giữa số khối và kích thước khối. Lớp $O(n)$ (tuyến tính) đại diện cho các thuật toán cần duyệt qua toàn bộ đầu vào một số lần không đổi, đây thường là độ phức tạp tốt nhất có thể vì ít nhất chúng ta phải đọc hết dữ liệu. Lớp $O(n \log n)$ rất phổ biến, thường gắn liền với các thuật toán sắp xếp hiệu quả hoặc các thuật toán sử dụng cấu trúc dữ liệu mà mỗi thao tác mất $O(\log n)$ thời gian.

Các lớp độ phức tạp đa thức cao hơn bao gồm $O(n^2)$ (bậc hai) và $O(n^3)$ (bậc ba), thường tương ứng với việc duyệt qua tất cả các cặp hoặc bộ ba phần tử trong đầu

vào. Cuối cùng, có những lớp độ phức tạp rất chậm như $O(2^n)$ (hàm mũ) và $O(n!)$ (giai thừa). Độ phức tạp $O(2^n)$ thường xuất hiện khi thuật toán phải duyệt qua tất cả các tập con của một tập hợp, trong khi $O(n!)$ liên quan đến việc duyệt qua tất cả các hoán vị. Các thuật toán này chỉ khả thi với kích thước đầu vào rất nhỏ.

Thuật toán Đa thức và Giới thiệu về NP-khó

Một thuật toán được gọi là thuật toán đa thức (polynomial algorithm) nếu độ phức tạp thời gian của nó không vượt quá $O(n^k)$ với k là một hằng số. Tất cả các lớp độ phức tạp chúng ta đã thảo luận, ngoại trừ $O(2^n)$ và $O(n!)$, đều thuộc loại đa thức. Trong thực tế, một thuật toán có độ phức tạp đa thức thường được coi là "hiệu quả", đặc biệt khi hằng số k nhỏ (thường là 2 hoặc 3). Hầu hết các thuật toán được trình bày trong lập trình thi đấu đều là thuật toán đa thức.

Tuy nhiên, có một lớp các bài toán quan trọng mà cho đến nay, chưa ai tìm ra được thuật toán giải quyết trong thời gian đa thức. Đây là những bài toán NP-khó (NP-hard problems). Cụm từ "chưa ai tìm ra" mang một ý nghĩa rất mạnh mẽ trong khoa học máy tính: nó không có nghĩa là không tồn tại, nhưng sau nhiều thập kỷ nghiên cứu của các nhà khoa học hàng đầu, vẫn chưa có một giải pháp hiệu quả nào được khám phá. Việc nhận diện một bài toán có thể là NP-khó giúp lập trình viên tránh lãng phí thời gian vào việc tìm kiếm một giải pháp đa thức không tồn tại và thay vào đó tập trung vào các phương pháp khác như thuật toán xấp xỉ hoặc duyệt toàn bộ với các ràng buộc nhỏ.

Ước tính Hiệu quả trong Thực tế

Phân tích độ phức tạp không chỉ là lý thuyết suông mà còn là một công cụ thực tiễn vô giá. Một quy tắc kinh nghiệm hữu ích là một máy tính hiện đại có thể thực hiện khoảng vài trăm triệu (10^8) phép tính trong một giây. Dựa vào đây, chúng ta có thể ước tính liệu một thuật toán có đủ nhanh cho một bài toán cụ thể hay không. Ví dụ, giả sử giới hạn thời gian là một giây và kích thước đầu vào là $n = 10^5$. Nếu thuật toán của bạn có độ phức tạp là $O(n^2)$, nó sẽ thực hiện khoảng $(10^5)^2 = 10^{10}$ phép tính. Con số này lớn hơn rất nhiều so với 10^8 , cho thấy thuật toán gần như chắc chắn sẽ quá chậm.

Ngược lại, từ giới hạn kích thước đầu vào, chúng ta có thể suy luận về độ phức tạp yêu cầu của thuật toán. Bảng dưới đây cung cấp một số ước tính hữu ích với giới hạn thời gian một giây:

- $n \leq 10$: $O(n!)$ có thể chấp nhận được.
- $n \leq 20$: $O(2^n)$ có thể chấp nhận được.
- $n \leq 500$: $O(n^3)$ có thể chấp nhận được.
- $n \leq 5000$: $O(n^2)$ có thể chấp nhận được.
- $n \leq 10^6$: Cần thuật toán $O(n \log n)$ hoặc $O(n)$.
- n rất lớn (ví dụ 10^{12}): Cần thuật toán $O(\log n)$ hoặc $O(1)$.

Bảng ước tính này là một kim chỉ nam cực kỳ quan trọng trong việc thiết kế giải thuật. Nếu bạn thấy n lên tới 10^5 , bạn ngay lập tức biết rằng mình cần tìm một giải pháp tuyến tính hoặc gần tuyến tính, và có thể loại bỏ ngay các ý tưởng dẫn đến độ phức tạp $O(n^2)$. Tuy nhiên, cần nhớ rằng Big O chỉ là một ước tính, nó ẩn đi các hằng số. Một thuật toán $O(n)$ có thể thực hiện $n/2$ phép tính hoặc $5n$ phép tính, và sự khác biệt này có thể ảnh hưởng đáng kể đến thời gian chạy thực tế, đôi khi quyết định việc thuật toán có qua được giới hạn thời gian hay không.

Điểm cốt lõi

- > Độ phức tạp thời gian, ký hiệu là $O(\dots)$ (Big O), ước tính thời gian chạy của thuật toán dưới dạng một hàm theo kích thước đầu vào n , bỏ qua các hằng số.
- > Khi thuật toán có nhiều pha, độ phức tạp tổng thể được quyết định bởi pha chậm nhất. Độ phức tạp của k vòng lặp lồng nhau thường là $O(n^k)$.
- > Các lớp độ phức tạp phổ biến xếp từ nhanh đến chậm bao gồm: $O(1)$, $O(\log n)$, $O(\sqrt{n})$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$, $O(2^n)$, $O(n!)$.
- > Thuật toán đa thức (ví dụ: $O(n^k)$) thường được coi là hiệu quả, trong khi các bài toán NP-khó hiện chưa có lời giải hiệu quả nào được biết đến.
- > Dựa vào giới hạn đầu vào (ví dụ $n \leq 10^6$), ta có thể suy luận độ phức tạp mục tiêu (thường là $O(n \log n)$ hoặc $O(n)$) để định hướng thiết kế giải thuật.

Câu hỏi ôn tập

1. Một thuật toán bao gồm ba bước tuần tự: bước đầu có độ phức tạp $O(n \log n)$, bước hai có độ phức tạp $O(n^2)$, và bước ba có độ phức tạp $O(n)$. Độ phức tạp tổng thể của thuật toán này là gì và tại sao?
2. Hãy giải thích sự khác biệt cơ bản giữa một thuật toán có độ phức tạp $O(\log n)$ và một thuật toán có độ phức tạp $O(n)$. Nêu một ví dụ về tình huống có thể dẫn đến mỗi loại độ phức tạp này.
3. Nếu một bài toán có kích thước đầu vào n lên tới 20 và thời gian giới hạn là 1 giây, bạn sẽ ưu tiên tìm kiếm giải thuật thuộc lớp độ phức tạp nào? Tại sao giải thuật $O(n^2)$ có thể không phù hợp?
4. Thế nào là một thuật toán đa thức? Tại sao các thuật toán có độ phức tạp $O(2^n)$ và $O(n!)$ không được coi là thuật toán đa thức?

Chương 4: Các Kỹ thuật Tìm kiếm và Thao tác Bit

Chào mừng bạn đến với chương 4, nơi chúng ta sẽ khám phá một trong những phương pháp giải quyết bài toán cơ bản nhưng vô cùng mạnh mẽ: tìm kiếm vét cạn. Đây là một kỹ thuật tổng quát, có thể áp dụng cho hầu hết mọi bài toán thuật toán bằng cách duyệt qua tất cả các giải pháp khả thi. Mặc dù đơn giản, tìm kiếm vét cạn là nền tảng quan trọng trước khi tiếp cận các kỹ thuật phức tạp hơn. Song song đó, chương này sẽ đi sâu vào thế giới của thao tác bit, một bộ công cụ cho phép chúng ta xử lý dữ liệu ở mức độ thấp nhất, mang lại hiệu quả đáng kinh ngạc và mở ra những cách tiếp cận độc đáo để giải quyết vấn đề, đặc biệt là trong việc biểu diễn và duyệt qua các không gian trạng thái nhỏ.

Mục tiêu học tập

- Trình bày được nguyên tắc hoạt động của tìm kiếm vét cạn và cách áp dụng để sinh tất cả các tập hợp con của một tập hợp.
- Giải thích được cách dùng biểu diễn bit của số nguyên để duyệt qua các tập hợp con một cách hiệu quả.
- Sử dụng thành thạo các toán tử thao tác bit (AND, OR, XOR, NOT, dịch bit) để giải quyết các bài toán con.
- Phân biệt được ưu và nhược điểm của tìm kiếm vét cạn so với các phương pháp tối ưu hóa khác.

Tìm kiếm vét cạn: Sinh tất cả tập hợp con bằng đệ quy

Tìm kiếm vét cạn (complete search), hay còn gọi là brute force, là một phương pháp giải quyết bài toán bằng cách sinh ra một cách có hệ thống tất cả các lời giải tiềm năng và sau đó kiểm tra từng lời giải để chọn ra lời giải tốt nhất hoặc đếm số lượng lời giải thỏa mãn. Ưu điểm lớn nhất của phương pháp này là tính đơn giản trong việc cài đặt và đảm bảo luôn tìm ra câu trả lời chính xác nếu tồn tại. Đây là kỹ thuật nên được cân nhắc đầu tiên khi bạn đối mặt với một bài toán mới, đặc biệt khi giới hạn của dữ liệu đầu vào đủ nhỏ để việc duyệt qua mọi khả năng không tốn quá nhiều thời gian. Nếu tìm kiếm vét cạn quá chậm, chúng ta mới cần đến các kỹ thuật nâng cao hơn như thuật toán tham lam hay quy hoạch động.

Một trong những bài toán kinh điển nhất để minh họa cho tìm kiếm vét cạn là sinh tất cả các tập hợp con của một tập hợp cho trước. Ví dụ, với tập hợp 0, 1, 2, có tổng cộng 8 tập hợp con: tập rỗng ($\emptyset$), 0, 1, 2, 0, 1, 0, 2, 1, 2, và 0, 1, 2. Một cách tiếp cận rất tự nhiên và thanh lịch để giải quyết bài toán này là sử dụng đệ quy. Ý tưởng cốt lõi là với mỗi phần tử trong tập hợp ban đầu, chúng ta có hai lựa chọn: hoặc đưa nó vào tập hợp con đang xây dựng, hoặc không. Bằng cách duyệt qua tất cả các phần tử và đưa ra quyết định tại mỗi bước, chúng ta sẽ tạo ra được tất cả các tập hợp con có thể.

Hãy xem xét một hàm đệ quy `search(k)` để sinh các tập con của tập 0, 1, ..., n-1. Hàm này sẽ duy trì một cấu trúc dữ liệu, chẳng hạn như một vector, để lưu trữ tập

con hiện tại. Khi `search(k)` được gọi, nó sẽ quyết định cho phần tử `k`. Đầu tiên, hàm sẽ gọi đệ quy `search(k+1)` để xử lý các phần tử còn lại mà không bao gồm `k`. Sau đó, nó thêm `k` vào tập con, rồi lại gọi đệ quy `search(k+1)` một lần nữa để xử lý các phần tử còn lại trong trường hợp có bao gồm `k`. Cuối cùng, nó loại bỏ `k` khỏi tập con để quay lui (backtrack), trả về trạng thái ban đầu cho các nhánh đệ quy khác. Quá trình này dừng lại khi `k` bằng `n`, nghĩa là tất cả các phần tử đã được xem xét và một tập hợp con hoàn chỉnh đã được hình thành. Toàn bộ quá trình này có thể được hình dung như một cây nhị phân, trong đó mỗi nút ở độ sâu `k` đại diện cho quyết định với phần tử `k`, và mỗi con đường từ gốc đến lá tương ứng với một tập hợp con duy nhất.

Kỹ thuật thao tác bit trong C++

Bên cạnh đệ quy, có một phương pháp khác cũng rất hiệu quả để sinh tập con, đó là dựa vào biểu diễn nhị phân của các số nguyên. Mỗi tập hợp con của một tập hợp `n` phần tử có thể được ánh xạ tới một chuỗi `n` bit, tương ứng với một số nguyên trong khoảng từ 0 đến $2^n - 1$. Trong chuỗi bit này, bit thứ `i` (tính từ phải sang trái, bắt đầu từ 0) bằng 1 có nghĩa là phần tử thứ `i` của tập hợp gốc được bao gồm trong tập con, và ngược lại. Ví dụ, với `n=5`, số 25 có biểu diễn nhị phân là 11001, tương ứng với tập con 0, 3, 4. Kỹ thuật này, thường được gọi là mặt nạ bit (bitmask), cho phép chúng ta duyệt qua tất cả các tập con bằng một vòng lặp đơn giản từ 0 đến $(1 < n) - 1$, với $1 < n$ tương đương 2^n . Trong mỗi lần lặp, giá trị của biến lặp chính là một mặt nạ bit đại diện cho một tập con.

Để hiểu sâu hơn về kỹ thuật này, chúng ta cần nắm vững các phép toán trên bit. Trong máy tính, mọi dữ liệu đều được lưu trữ dưới dạng các bit 0 và 1. Một số nguyên 32-bit trong C++ (kiểu `int`) được biểu diễn bằng một chuỗi 32 bit. Ví dụ, số 43 có biểu diễn 32-bit là 0000000000000000000000000101011. Giá trị của nó được tính bằng công thức $b_k 2^k + \dots + b_1 2^1 + b_0 2^0$. Với số 43, ta có $1 \text{ imes } 2^5 + 0 \text{ imes } 2^4 + 1 \text{ imes } 2^3 + 0 \text{ imes } 2^2 + 1 \text{ imes } 2^1 + 1 \text{ imes } 2^0 = 32 + 8 + 2 + 1 = 43$. Các số nguyên thường được biểu diễn dưới dạng có dấu (signed), nghĩa là bit đầu tiên dùng để chỉ dấu (0 cho số không âm, 1 cho số âm), cho phép biểu diễn cả số dương và số âm.

C++ cung cấp các toán tử để thao tác trực tiếp trên các bit này. Toán tử AND (&) trả về 1 tại các vị trí mà cả hai toán hạng đều có bit 1. Ví dụ, 22 (10110) & 26 (11010) cho kết quả 18 (10010). Một ứng dụng phổ biến là kiểm tra một số `x` có chẵn hay không bằng cách tính `x & 1` (kết quả là 0 nếu chẵn, 1 nếu lẻ). Toán tử OR (|) trả về 1 nếu ít nhất một trong hai toán hạng có bit 1. Ví dụ, 22 | 26 cho kết quả 30 (11110). Toán tử XOR (^) trả về 1 nếu chỉ một trong hai toán hạng có bit 1. Ví dụ, 22 ^ 26 cho kết quả 12 (01100).

Ngoài ra, còn có toán tử NOT (~) đảo ngược tất cả các bit của một số (ví dụ, ~29 là -30 trong hệ 32-bit) và các toán tử dịch bit. Phép dịch trái `x << k` thêm `k` bit 0 vào cuối số, tương đương với việc nhân `x` với 2^k . Ngược lại, phép dịch phải `x >> k` loại bỏ `k` bit cuối cùng, tương đương với việc chia nguyên `x` cho 2^k . Ví dụ, 14 (1110) << 2 cho kết quả 56 (111000). Những toán tử này là công cụ nền tảng, cực kỳ hiệu quả

cho nhiều tác vụ trong lập trình thi đấu.

Các hàm `_builtin` để đếm bit

Các tài liệu được cung cấp không chứa thông tin chi tiết về các hàm `_builtin` của trình biên dịch GCC/Clang, chẳng hạn như `_builtin_popcount` (đếm số bit 1), `_builtin_clz` (đếm số bit 0 ở đầu), hay `_builtin_ctz` (đếm số bit 0 ở cuối). Đây là những hàm được tối ưu hóa cao ở cấp độ phần cứng và rất hữu ích trong lập trình thi đấu, nhưng không được đề cập trong phạm vi ngữ cảnh này.

Mở rộng tìm kiếm nhị phân: Tìm giá trị lớn nhất

Nội dung về kỹ thuật tìm kiếm nhị phân, đặc biệt là ứng dụng mở rộng của nó để tìm kiếm câu trả lời tối ưu (ví dụ: tìm giá trị lớn nhất thỏa mãn một điều kiện nào đó), không được trình bày trong các đoạn văn bản nguồn đã cho. Kỹ thuật này, thường được gọi là 'tìm kiếm nhị phân trên kết quả' (binary search on the answer), là một công cụ mạnh mẽ nhưng nằm ngoài phạm vi của tài liệu tham khảo cho chương này.

Điểm cốt lõi

- > Tìm kiếm vét cạn là một phương pháp tổng quát, duyệt qua mọi khả năng để tìm ra lời giải, đảm bảo tính đúng đắn nhưng có thể chậm về mặt thời gian.
- > Có hai cách phổ biến để sinh tất cả các tập hợp con của một tập hợp n phần tử: dùng đệ quy hoặc dùng mặt nạ bit (bitmask) lặp từ 0 đến $2^n - 1$.
- > Mỗi số nguyên có thể được xem như một 'mặt nạ bit', trong đó mỗi bit đại diện cho sự hiện diện hoặc vắng mặt của một phần tử trong một tập hợp.
- > Thao tác bit cho phép thực hiện các phép toán logic và số học ở mức độ bit, thường nhanh hơn nhiều so với các phép toán thông thường.
- > Các toán tử bit như `&`, `|`, `^`, `~`, `<<`, `>>` là công cụ mạnh mẽ để kiểm tra thuộc tính, bật/tắt bit, và thực hiện phép nhân/chia cho lũy thừa của 2.

Câu hỏi ôn tập

1. Sự khác biệt cơ bản giữa phương pháp sinh tập con bằng đệ quy và bằng mặt nạ bit là gì? Khi nào bạn sẽ ưu tiên sử dụng phương pháp này hơn phương pháp kia?
2. Làm thế nào để sử dụng toán tử AND (`&`) và toán tử dịch trái (`<<`) để kiểm tra xem bit thứ k của một số nguyên x có đang được bật (bằng 1) hay không?
3. Giải thích tại sao phép dịch trái $x \ll k$ tương đương với $x * 2^k$ và phép dịch phải $x \gg k$ tương đương với $x / 2^k$ (làm tròn xuống).
4. Một bài toán yêu cầu bạn tìm một tập con có tổng các phần tử bằng một giá trị cho trước trong một mảng gồm 20 số nguyên. Phương pháp tìm kiếm vét cạn có phù hợp để giải quyết bài toán này không? Tại sao?

Chương 5: Quy hoạch động trên Tập hợp con

Trong lập trình thi đấu, chúng ta thường xuyên gặp các bài toán yêu cầu xử lý thông tin liên quan đến các tập hợp con của một tập hợp cho trước. Một cách tiếp cận tự nhiên là duyệt qua tất cả các cặp tập hợp, nhưng phương pháp này nhanh chóng trở nên quá chậm khi kích thước của tập hợp ban đầu tăng lên. Để giải quyết vấn đề này một cách hiệu quả, chúng ta sử dụng một kỹ thuật quy hoạch động nâng cao, thường được biết đến với tên gọi "Sum over Subsets" (SOS DP), hay Quy hoạch động trên Tập hợp con. Kỹ thuật này tận dụng sức mạnh của biểu diễn tập hợp bằng mặt nạ bit (bitmask) để xây dựng lời giải một cách có hệ thống, giảm đáng kể độ phức tạp tính toán và mở ra khả năng giải quyết một lớp bài toán rộng lớn hơn.

Mục tiêu học tập

- Hiểu và trình bày được bài toán "tổng trên các tập con" (Sum over Subsets).
- Giải thích được ý tưởng quy hoạch động và công thức truy hồi của kỹ thuật SOS DP.
- Cài đặt được thuật toán SOS DP một cách hiệu quả bằng mảng và các phép toán trên bit.
- Phân tích được độ phức tạp thời gian của thuật toán là $O(n 2^n)$.

Bài toán Tổng trên các Tập hợp con

Bài toán nền tảng cho kỹ thuật này có thể phát biểu như sau: Cho một tập hợp gồm n phần tử, thường được đánh số từ 0 đến $n-1$. Với mỗi tập hợp con S trong số 2^n tập con có thể có, chúng ta được cung cấp một giá trị ban đầu, ký hiệu là $value[S]$. Nhiệm vụ của chúng ta là tính toán, với mỗi tập con S , một giá trị mới là $sum[S]$, được định nghĩa là tổng của tất cả các $value[A]$ với A là một tập hợp con của S (bao gồm cả tập rỗng và chính S). Công thức toán học của nó là: $sum[S] = \sum (A \subseteq S) value[A]$.

Để làm rõ hơn, hãy xét một ví dụ với $n=3$. Các phần tử là 0, 1, 2. Giả sử chúng ta có các giá trị ban đầu cho mỗi tập con (biểu diễn bằng bitmask): $value[;]=3$, $value[0]=1$, $value[1]=4$, $value[0,1]=5$, $value[2]=5$, $value[0,2]=1$, $value[1,2]=3$, $value[0,1,2]=3$. Bây giờ, nếu chúng ta muốn tính $sum[0,2]$, chúng ta cần tìm tất cả các tập con của 0,2. Các tập con này là: tập rỗng ($\emptyset$), 0, 2, và 0,2. Theo định nghĩa, $sum[0,2]$ sẽ là tổng các giá trị tương ứng: $sum[0,2] = value[;] + value[0] + value[2] + value[0,2] = 3 + 1 + 5 + 1 = 10$.

Cách tiếp cận đơn giản nhất để giải quyết bài toán này là duyệt qua từng tập con S trong số 2^n tập con. Với mỗi S , ta lại duyệt qua tất cả 2^n tập con A khác và kiểm tra xem A có phải là tập con của S hay không. Nếu có, ta cộng $value[A]$ vào $sum[S]$. Một cách kiểm tra A ext là tập con của S là $(A \& S) == A$. Cách làm này đòi hỏi hai vòng lặp lồng nhau duyệt qua 2^n phần tử, dẫn đến độ phức tạp thời gian là $O(2^n \text{ imes } 2^n) = O(4^n)$. Một cải tiến nhỏ là với mỗi S , ta chỉ duyệt qua các tập con của chính nó, nhưng tổng độ phức tạp vẫn ở mức $O(3^n)$, quá chậm cho các giá trị n thường gặp trong các cuộc thi (ví dụ n ext lên đến 20). Rõ ràng, chúng ta cần một phương pháp hiệu quả

hơn.

Ý tưởng Quy hoạch động và Công thức Truy hồi

Thay vì tính toán mỗi $\text{sum}[S]$ một cách độc lập, ý tưởng của quy hoạch động là xây dựng các giá trị này một cách từ từ, có hệ thống. Chúng ta sẽ định nghĩa một trạng thái quy hoạch động dựa trên việc giới hạn các phần tử được phép có mặt trong các tập con. Cụ thể, ta định nghĩa $\text{dp}[S][k]$ là tổng các $\text{value}[A]$ với A là tập con của S ($A \subseteq S$), nhưng với một điều kiện bổ sung: các phần tử trong A phải là một tập con của các phần tử trong S mà chỉ được lấy từ tập $0, 1, \dots, k$. Một cách diễn đạt khác tương đương từ tài liệu gốc là $\text{dp}[S][k]$ là tổng giá trị của các tập con của S với ràng buộc rằng chỉ các phần tử từ 0 đến k mới có thể bị loại bỏ khỏi S .

Với định nghĩa này, chúng ta có thể xây dựng một công thức truy hồi. Trạng thái cơ sở của quy hoạch động là khi không có phần tử nào được phép loại bỏ, tức là $k = -1$. Trong trường hợp này, tập con duy nhất của S thỏa mãn điều kiện là chính nó. Do đó, ta có: $\text{dp}[S][-1] = \text{value}[S]$. Đây là điểm khởi đầu của chúng ta.

Bây giờ, hãy xem xét bước chuyển trạng thái từ $k-1$ sang k . Để tính $\text{dp}[S][k]$, chúng ta tập trung vào phần tử k . Có hai trường hợp xảy ra:

1. **Phần tử k không thuộc tập S** ($k \notin S$):

Nếu k không có trong S , nó cũng không thể có trong bất kỳ tập con A nào của S . Do đó, việc cho phép loại bỏ thêm phần tử k không làm thay đổi tập các tập con A hợp lệ. Vì vậy, $\text{dp}[S][k] = \text{dp}[S][k-1]$.

2. **Phần tử k thuộc tập S** ($k \in S$):

Khi k có trong S , các tập con A của S (với điều kiện loại bỏ chỉ trong $0, \dots, k$) có thể được chia thành hai nhóm: những tập con không chứa k và những tập con chứa k . Nhóm không chứa k thực chất là các tập con của $S \setminus \{k\}$ với điều kiện loại bỏ trong $0, \dots, k-1$, tổng giá trị của chúng là $\text{dp}[S \setminus \{k\}][k-1]$. Nhóm chứa k có thể được xem là được tạo ra từ các tập con của S với điều kiện loại bỏ trong $0, \dots, k-1$, tổng giá trị của chúng là $\text{dp}[S][k-1]$. Do đó, tổng cộng chúng ta có: $\text{dp}[S][k] = \text{dp}[S][k-1] + \text{dp}[S \setminus \{k\}][k-1]$.

Sau khi thực hiện quy hoạch động cho tất cả các phần tử từ 0 đến $n-1$, giá trị $\text{dp}[S][n-1]$ sẽ là tổng giá trị của tất cả các tập con A của S mà không có ràng buộc nào về việc loại bỏ phần tử. Điều này chính xác là định nghĩa của $\text{sum}[S]$ mà chúng ta cần tìm. Do đó, $\text{sum}[S] = \text{dp}[S][n-1]$.

Cách cài đặt hiệu quả với Mảng và Bitmask

Công thức truy hồi ở trên có vẻ phức tạp với ba chiều (S , k và A), nhưng có một cách cài đặt cực kỳ thông minh và hiệu quả chỉ sử dụng một mảng một chiều. Đầu tiên, chúng ta cần biểu diễn các tập hợp. Một cách hiệu quả để làm điều này là sử dụng mặt nạ bit (bitmask). Một số nguyên n bit có thể đại diện cho bất kỳ tập con nào của tập $0, 1, \dots, n-1$. Bit thứ i (tính từ phải sang, bắt đầu từ 0) của số nguyên bằng 1 nếu phần tử i có trong tập hợp, và bằng 0 nếu ngược lại. Ví dụ, với $n=5$, tập $0, 3, 4$ được biểu diễn bởi số nhị phân 11001 , tương ứng với số nguyên 25 . Các phép toán trên tập hợp như hợp, giao có thể được thực hiện nhanh chóng bằng các phép toán bit như OR ($|$) và AND ($\&$).

Để cài đặt SOS DP, chúng ta sử dụng một mảng duy nhất, gọi là dp , có kích thước 2^n . $dp[mask]$ sẽ lưu trữ giá trị tổng đang được tính cho tập hợp tương ứng với $mask$. Ban đầu, ta khởi tạo mảng này với các giá trị gốc: `for (int mask = 0; mask < (1<<n); mask++) dp[mask] = value[mask];`. Bước này tương ứng với trạng thái cơ sở $dp[S][-1] = value[S]$.

Tiếp theo, chúng ta thực hiện quy hoạch động thông qua hai vòng lặp lồng nhau. Vòng lặp bên ngoài duyệt qua từng phần tử i từ 0 đến $n-1$. Vòng lặp bên trong duyệt qua tất cả các mặt nạ $mask$ từ 0 đến 2^n-1 . Bên trong vòng lặp, chúng ta áp dụng công thức cập nhật:

```
for (int i = 0; i < n; i++)
for (int mask = 0; mask < (1<<n); mask++)
if (mask & (1<<i))
dp[mask] += dp[mask ^ (1<<i)];
```

..

Để hiểu tại sao đoạn mã này hoạt động, hãy phân tích nó. Vòng lặp ngoài qua i tương ứng với chiều k trong công thức truy hồi. Khi bắt đầu vòng lặp i , mảng dp đang lưu các giá trị $dp[S][i-1]$. Câu lệnh `if (mask & (1<<i))` kiểm tra xem phần tử i có thuộc tập hợp $mask$ hay không. Nếu có, nó tương ứng với trường hợp $k \in S$. Khi đó, ta cập nhật $dp[mask]$ bằng cách cộng thêm $dp[mask \wedge (1<<i)]$. $mask \wedge (1<<i)$ là mặt nạ của tập hợp $mask$ nhưng không có phần tử i (tức là $S \text{ ext } i$). Tại thời điểm này, $dp[mask]$ đang giữ $dp[S][i-1]$ và $dp[mask \wedge (1<<i)]$ đang giữ $dp[S \text{ ext } i][i-1]$. Do đó, phép cộng $dp[mask] += dp[mask \wedge (1<<i)]$ chính là thực hiện $dp[S][i] = dp[S][i-1] + dp[S \text{ ext } i][i-1]$. Nếu phần tử i không thuộc $mask$, không có gì được thực hiện, tương ứng với $dp[S][i] = dp[S][i-1]$. Sau khi vòng lặp i kết thúc, mảng dp đã được cập nhật để chứa các giá trị $dp[S][i]$. Việc tái sử dụng cùng một mảng dp là hoàn toàn chính xác vì để tính $dp[S][i]$, chúng ta chỉ cần các giá trị từ bước $i-1$, vốn đã có sẵn trong mảng.

Phân tích Độ phức tạp

Độ phức tạp của thuật toán SOS DP có thể được phân tích một cách trực tiếp từ cách cài đặt hiệu quả. Thuật toán bao gồm một bước khởi tạo và một cặp vòng lặp lồng nhau. Bước khởi tạo duyệt qua 2^n mặt nạ để sao chép các giá trị từ value sang dp, tốn thời gian $O(2^n)$.

Phần chính của thuật toán là hai vòng lặp lồng nhau. Vòng lặp bên ngoài chạy n lần, cho biến i từ 0 đến $n-1$. Vòng lặp bên trong chạy 2^n lần, cho biến mask từ 0 đến 2^n-1 . Các thao tác bên trong vòng lặp, bao gồm kiểm tra bit (&), phép XOR (^), và phép cộng, đều là các phép toán cơ bản và tốn thời gian không đổi, tức là $O(1)$. Do đó, tổng thời gian thực thi của hai vòng lặp này là tích của số lần lặp của chúng: $n \text{ imes } 2^n$. Tổng hợp lại, độ phức tạp thời gian của toàn bộ thuật toán là $O(2^n + n 2^n) = O(n 2^n)$.

Để thấy rõ sự cải thiện vượt bậc, hãy so sánh với độ phức tạp $O(3^n)$ của phương pháp duyệt ngây thơ. Giả sử $n=20$. Với SOS DP, số phép tính vào khoảng $20 \text{ imes } 2^{20} \text{ ext } \approx 20 \text{ imes } 10^6$, một con số hoàn toàn khả thi trên các máy tính hiện đại. Ngược lại, với phương pháp ngây thơ, số phép tính là $3^{20} \text{ ext } \approx 3.5 \text{ imes } 10^9$, lớn hơn rất nhiều và gần như chắc chắn sẽ gây ra lỗi "Time Limit Exceeded" trong một cuộc thi. Sự khác biệt này cho thấy sức mạnh của việc chuyển đổi một bài toán tổ hợp sang quy hoạch động.

Về độ phức tạp không gian, thuật toán yêu cầu một mảng dp (và có thể cả mảng value ban đầu) để lưu trữ giá trị cho mỗi trong số 2^n tập hợp con. Do đó, độ phức tạp không gian là $O(2^n)$. Đây là một sự đánh đổi hợp lý: chúng ta sử dụng thêm bộ nhớ để đạt được tốc độ tính toán nhanh hơn đáng kể.

Điểm cốt lõi

- > Quy hoạch động trên tập con (SOS DP) là kỹ thuật hiệu quả để giải các bài toán tính toán trên tất cả các tập con của một tập hợp.
- > Kỹ thuật này biểu diễn các tập hợp bằng mặt nạ bit (bitmask) và sử dụng một mảng kích thước $O(2^n)$ để lưu trữ các giá trị quy hoạch động.
- > Công thức truy hồi cốt lõi dựa trên việc xét từng phần tử một và tính tổng cho tập hợp hiện tại dựa trên tổng của chính nó và tổng của tập hợp con thiếu đi phần tử đó.
- > Cách cài đặt lặp hiệu quả với hai vòng lặp lồng nhau đạt được độ phức tạp thời gian $O(n 2^n)$, vượt trội so với các phương pháp duyệt ngây thơ.
- > Việc hiểu rõ cách trạng thái quy hoạch động $dp[mask]$ được cập nhật qua từng bước lặp của phần tử i là chìa khóa để nắm vững thuật toán.

Câu hỏi ôn tập

1. Trình bày sự khác biệt về độ phức tạp giữa phương pháp duyệt ngây thơ và phương pháp quy hoạch động để giải bài toán tổng trên các tập con. Tại sao SOS DP lại hiệu quả hơn?

2. Trong cách cài đặt SOS DP, tại sao vòng lặp ngoài phải duyệt qua các phần tử i từ 0 đến $n-1$, còn vòng lặp trong duyệt qua các mặt nạ mask? Thứ tự của hai vòng lặp này có thể hoán đổi được không và tại sao?
3. Làm thế nào để sửa đổi thuật toán SOS DP để tính, với mỗi tập S , tổng các $\text{value}[A]$ trên tất cả các tập cha A (tức là $S \text{ ext} \subseteq A$)?
4. Nếu bài toán yêu cầu tìm giá trị lớn nhất (thay vì tổng) trên các tập con, tức $\max[S] = \max_{A \subseteq S} \text{value}[A]$, thuật toán SOS DP có thể được điều chỉnh như thế nào?

Chương 6: Lý thuyết Đồ thị Cơ bản và Thuật toán Tìm chu trình

Đồ thị là một trong những cấu trúc dữ liệu cơ bản và mạnh mẽ nhất trong khoa học máy tính, cho phép chúng ta mô hình hóa và giải quyết vô số bài toán liên quan đến các mối quan hệ và mạng lưới, từ mạng xã hội, hệ thống giao thông đến lập lịch công việc. Việc nắm vững các khái niệm nền tảng của lý thuyết đồ thị là bước đệm không thể thiếu cho bất kỳ lập trình viên thi đấu nào. Chương này sẽ trang bị cho bạn những thuật ngữ cốt lõi như đỉnh, cạnh, đường đi, chu trình, và tính liên thông. Đặc biệt, chúng ta sẽ đi sâu vào một thuật toán kinh điển và cực kỳ hiệu quả về mặt bộ nhớ để phát hiện chu trình: thuật toán của Floyd, một công cụ mạnh mẽ trong kho vũ khí của bạn.

Mục tiêu học tập

- Định nghĩa và phân biệt được các thuật ngữ cơ bản của lý thuyết đồ thị như đỉnh, cạnh, đường đi, và chu trình.
- Giải thích khái niệm về tính liên thông, thành phần liên thông mạnh và cây trong đồ thị.
- Trình bày và áp dụng được thuật toán tìm chu trình của Floyd để phát hiện chu trình, tìm điểm bắt đầu và tính độ dài chu trình với hiệu quả bộ nhớ tối ưu.

Các Thuật Ngữ Đồ Thị Cơ Bản

Để bắt đầu hành trình khám phá lý thuyết đồ thị, chúng ta cần làm quen với bộ từ vựng nền tảng của nó. Một đồ thị (graph) là một cấu trúc toán học bao gồm một tập hợp các đỉnh (nodes hoặc vertices) và một tập hợp các cạnh (edges) nối các cặp đỉnh lại với nhau. Bạn có thể hình dung các đỉnh như những thành phố và các cạnh như những con đường nối chúng. Các cạnh có thể có hướng (directed), giống như đường một chiều, hoặc vô hướng (undirected), giống như đường hai chiều. Trong nhiều bài toán, các cạnh còn có thể được gán một trọng số (weight), đại diện cho chi phí, khoảng cách, hoặc thời gian di chuyển giữa hai đỉnh.

Một đường đi (path) trong đồ thị là một chuỗi các đỉnh mà mỗi cặp đỉnh kế nhau trong chuỗi được nối với nhau bằng một cạnh. Độ dài của một đường đi thường được tính bằng số cạnh trên đường đi đó, hoặc tổng trọng số của các cạnh nếu đồ thị có trọng số. Khái niệm đường đi là trung tâm của nhiều thuật toán, chẳng hạn như tìm đường đi ngắn nhất giữa hai điểm. Khi một đường đi bắt đầu và kết thúc tại cùng một đỉnh, và đi qua ít nhất một đỉnh khác, nó tạo thành một chu trình (cycle). Việc phát hiện chu trình là một bài toán quan trọng, vì sự tồn tại của chu trình có thể ảnh hưởng lớn đến tính chất của đồ thị và thuật toán áp dụng trên nó.

Ví dụ, hãy tưởng tượng một mạng lưới các chuyến bay giữa các thành phố. Mỗi thành phố là một đỉnh, mỗi chuyến bay thẳng là một cạnh có hướng. Một chuỗi các chuyến bay nối tiếp nhau từ thành phố A đến thành phố B tạo thành một đường đi. Nếu bạn

có thể bay từ Hà Nội, qua Bangkok, đến Singapore, rồi lại bay thẳng về Hà Nội, bạn đã tạo ra một chu trình. Trong các bài toán lập lịch, một chu trình có thể đại diện cho một tình huống bế tắc (deadlock), nơi các tiến trình chờ đợi lẫn nhau theo một vòng tròn và không tiến trình nào có thể tiếp tục. Do đó, việc hiểu và xác định các cấu trúc này là kỹ năng cơ bản nhưng vô cùng quan trọng.

Tính Liên Thông và Thành Phần Liên Thông Mạnh

Tính liên thông mô tả mức độ kết nối của một đồ thị. Trong một đồ thị vô hướng, hai đỉnh được gọi là liên thông nếu có ít nhất một đường đi giữa chúng. Toàn bộ đồ thị được gọi là liên thông nếu mọi cặp đỉnh trong đồ thị đều liên thông với nhau. Tuy nhiên, đối với đồ thị có hướng, khái niệm này trở nên phức tạp hơn. Chỉ vì có đường đi từ đỉnh A đến đỉnh B không có nghĩa là sẽ có đường đi ngược lại từ B về A. Điều này dẫn đến một định nghĩa chặt chẽ hơn: tính liên thông mạnh (strong connectivity).

Một đồ thị có hướng được gọi là liên thông mạnh nếu với mọi cặp đỉnh (u, v) trong đồ thị, luôn tồn tại một đường đi từ u đến v và một đường đi từ v đến u . Điều này đảm bảo rằng từ bất kỳ điểm nào trong đồ thị, bạn đều có thể đến được mọi điểm khác. Các đồ thị không hoàn toàn liên thông mạnh có thể được phân rã thành các thành phần liên thông mạnh (Strongly Connected Components - SCCs). Mỗi SCC là một tập hợp con tối đại các đỉnh mà trong đó, mọi đỉnh đều có thể đến được từ mọi đỉnh khác. Bên trong một SCC, tính liên thông mạnh được đảm bảo.

Một ứng dụng kinh điển của thành phần liên thông mạnh là trong việc giải bài toán Thỏa mãn Mệnh đề 2-SAT. Một biểu thức 2-SAT bao gồm các mệnh đề logic, mỗi mệnh đề có dạng $(a \vee b)$, trong đó a và b là các biến logic hoặc phủ định của chúng. Bài toán yêu cầu tìm một phép gán giá trị (đúng/sai) cho các biến để toàn bộ biểu thức là đúng. Ta có thể chuyển biểu thức này thành một đồ thị có hướng, nơi mỗi biến x_i và phủ định của nó $\neg x_i$ là các đỉnh. Một mệnh đề $(a \vee b)$ tương đương với hai cạnh có hướng: $(\neg a \rightarrow b)$ và $(\neg b \rightarrow a)$. Biểu thức có lời giải khi và chỉ khi không có biến x_i nào mà cả x_i và $\neg x_i$ cùng thuộc một thành phần liên thông mạnh. Nếu chúng cùng một SCC, điều đó có nghĩa là x_i suy ra $\neg x_i$ và ngược lại, tạo ra một mâu thuẫn logic không thể giải quyết.

Cây: Một Dạng Đồ Thị Đặc Biệt

Trong thế giới đồ thị, cây (tree) là một trong những cấu trúc quan trọng và phổ biến nhất. Một cây được định nghĩa là một đồ thị liên thông và không có chu trình (acyclic). Hai đặc điểm này tạo nên những tính chất độc đáo và hữu ích. Vì không có chu trình, đường đi giữa bất kỳ hai đỉnh nào trong một cây là duy nhất. Vì liên thông, mọi đỉnh đều có thể kết nối với mọi đỉnh khác.

Một hệ quả trực tiếp và cũng là một định nghĩa tương đương của cây là mối quan hệ đặc biệt giữa số đỉnh và số cạnh. Một đồ thị có n đỉnh là một cây khi và chỉ khi nó liên thông và có đúng $n-1$ cạnh. Nếu một đồ thị liên thông có ít hơn $n-1$ cạnh, nó không thể kết nối tất cả các đỉnh. Ngược lại, nếu nó có nhiều hơn $n-1$ cạnh, nó chắc chắn sẽ

tạo ra ít nhất một chu trình. Mỗi quan hệ này là một công cụ kiểm tra nhanh và mạnh mẽ để xác định xem một đồ thị có phải là cây hay không.

Để dễ hình dung, hãy nghĩ về sơ đồ tổ chức của một công ty. Giám đốc điều hành ở trên cùng (gốc cây), các trưởng phòng báo cáo cho giám đốc, và nhân viên báo cáo cho các trưởng phòng. Đây là một cấu trúc cây điển hình: nó liên thông (mọi người đều kết nối với giám đốc thông qua một chuỗi quản lý) và không có chu trình (không có chuyện một nhân viên lại là quản lý của sếp mình). Việc thêm một cạnh mới vào cây, ví dụ như một nhân viên báo cáo cho hai người quản lý khác nhau, sẽ ngay lập tức tạo ra một chu trình. Ngược lại, việc xóa đi một cạnh bất kỳ sẽ làm cây mất tính liên thông, chia nó thành hai cây con riêng biệt.

Thuật Toán Tìm Chu Trình Của Floyd: Con Rùa và Con Thỏ

Khi xử lý các cấu trúc dữ liệu như danh sách liên kết hoặc đồ thị hàm (functional graph - mỗi đỉnh có đúng một cạnh đi ra), một bài toán thường gặp là phát hiện xem có tồn tại chu trình hay không. Một cách tiếp cận đơn giản là duyệt qua các đỉnh và lưu lại những đỉnh đã thăm vào một tập hợp; nếu gặp lại một đỉnh đã có trong tập hợp, ta đã tìm thấy một chu trình. Tuy nhiên, phương pháp này đòi hỏi bộ nhớ $O(n)$ để lưu trữ các đỉnh đã thăm, điều này có thể không khả thi với các tập dữ liệu lớn.

Thuật toán tìm chu trình của Floyd, thường được ví von là thuật toán "con rùa và con thỏ", cung cấp một giải pháp xuất sắc với độ phức tạp thời gian $O(n)$ nhưng chỉ sử dụng bộ nhớ $O(1)$. Ý tưởng cốt lõi là sử dụng hai con trỏ, a (con rùa) và b (con thỏ), cùng xuất phát từ một đỉnh x . Trong mỗi bước, con trỏ a di chuyển một bước ($a = \text{succ}(a)$), trong khi con trỏ b di chuyển hai bước ($b = \text{succ}(\text{succ}(b))$). Nếu trong đồ thị không có chu trình, con trỏ b sẽ nhanh chóng đến cuối (giá trị null). Tuy nhiên, nếu có một chu trình, con trỏ b sẽ đi vào chu trình trước và bắt đầu chạy vòng quanh. Con trỏ a sau đó cũng sẽ đi vào chu trình. Vì b di chuyển nhanh hơn a một bước trong mỗi lượt, khoảng cách giữa chúng giảm đi 1 mỗi khi chúng cùng ở trong chu trình. Do đó, b chắc chắn sẽ "bắt kịp" và gặp a tại một điểm nào đó bên trong chu trình.

Sau khi hai con trỏ gặp nhau lần đầu tiên (tại một đỉnh p), chúng ta có thể thực hiện pha thứ hai để tìm ra đỉnh bắt đầu của chu trình. Ta giữ con trỏ b tại vị trí gặp nhau p , và đặt lại con trỏ a về đỉnh xuất phát x . Bây giờ, cho cả hai con trỏ a và b cùng di chuyển từng bước một ($a = \text{succ}(a)$, $b = \text{succ}(b)$). Điểm mà chúng gặp nhau lần thứ hai chính là đỉnh đầu tiên của chu trình. Điều này được đảm bảo bởi tính chất toán học rằng khoảng cách từ đỉnh đầu đến điểm gặp nhau đầu tiên bằng khoảng cách từ điểm bắt đầu chu trình đến điểm gặp nhau đó.

Cuối cùng, khi đã xác định được đỉnh bắt đầu chu trình (gọi là first), ta có thể dễ dàng tính độ dài của nó. Giữ một con trỏ ở first , cho một con trỏ thứ hai (b) bắt đầu từ đỉnh tiếp theo ($\text{succ}(\text{first})$) và di chuyển từng bước một cho đến khi nó quay trở lại first . Số bước di chuyển chính là độ dài của chu trình. Toàn bộ quy trình, từ phát hiện, tìm điểm bắt đầu, đến tính độ dài, đều được thực hiện một cách hiệu quả mà không cần thêm không gian lưu trữ phụ thuộc vào kích thước đồ thị. Đây là một ví dụ tuyệt vời

về sự tinh tế và sức mạnh của thuật toán.

Điểm cốt lõi

- > Đồ thị là một cấu trúc toán học gồm các đỉnh (nodes) và cạnh (edges) dùng để mô hình hóa các mối quan hệ. Các khái niệm cốt lõi bao gồm đường đi (path) và chu trình (cycle).
- > Tính liên thông mạnh là một thuộc tính quan trọng của đồ thị có hướng, đòi hỏi phải có đường đi từ mọi đỉnh đến mọi đỉnh khác. Các đỉnh có thể được nhóm thành các thành phần liên thông mạnh.
- > Cây là một loại đồ thị đặc biệt: liên thông, không có chu trình, và luôn có $n-1$ cạnh với n đỉnh.
- > Thuật toán tìm chu trình của Floyd (còn gọi là thuật toán "con rùa và con thỏ") sử dụng hai con trỏ với tốc độ khác nhau để phát hiện chu trình trong thời gian $O(n)$ và bộ nhớ $O(1)$.
- > Thuật toán của Floyd không chỉ phát hiện sự tồn tại của chu trình mà còn có thể xác định chính xác đỉnh bắt đầu và độ dài của chu trình đó qua các pha xử lý tuần tự.

Câu hỏi ôn tập

1. Sự khác biệt cơ bản giữa một đồ thị liên thông thông thường và một đồ thị liên thông mạnh là gì? Tại sao khái niệm liên thông mạnh lại đặc biệt quan trọng đối với đồ thị có hướng?
2. Một đồ thị có 10 đỉnh và 8 cạnh. Đồ thị này có thể là một cây không? Tại sao?
3. Hãy giải thích nguyên lý hoạt động của thuật toán tìm chu trình của Floyd. Tại sao việc con trỏ nhanh di chuyển hai bước và con trỏ chậm di chuyển một bước lại đảm bảo chúng sẽ gặp nhau bên trong một chu trình?
4. Trong pha thứ hai của thuật toán Floyd để tìm điểm bắt đầu chu trình, tại sao việc đặt lại một con trỏ về điểm xuất phát và cho cả hai di chuyển cùng tốc độ lại giúp tìm ra đúng điểm bắt đầu?

Chương 7: Thành phần Liên thông mạnh và Truy vấn trên Cây

Trong lĩnh vực lý thuyết đồ thị, việc phân tích khả năng kết nối giữa các đỉnh là một bài toán nền tảng. Đối với đồ thị có hướng, khái niệm này được nâng lên một tầm cao mới với "tính liên thông mạnh", không chỉ đòi hỏi có đường đi giữa hai đỉnh mà là một sự kết nối hai chiều chặt chẽ. Việc xác định các "thành phần liên thông mạnh" (Strongly Connected Components - SCC) giúp chúng ta phân rã một đồ thị phức tạp thành các khối cấu trúc cơ bản, hé lộ một cấu trúc sâu sắc hơn dưới dạng đồ thị không chu trình. Chương này sẽ đi sâu vào thuật toán Kosaraju, một phương pháp hiệu quả để tìm các thành phần này, và khám phá ứng dụng mạnh mẽ của nó trong việc giải quyết bài toán logic 2-SAT. Ngoài ra, chương cũng sẽ giới thiệu một lớp bài toán quan trọng khác trên cây: truy vấn tổ tiên, một kỹ năng cần thiết để xử lý hiệu quả các cấu trúc phân cấp.

Mục tiêu học tập

- Định nghĩa và xác định được các thành phần liên thông mạnh trong một đồ thị có hướng.
- Trình bày và áp dụng được thuật toán Kosaraju để phân rã đồ thị thành các thành phần liên thông mạnh.
- Hiểu và sử dụng khái niệm đồ thị thành phần để giải quyết các bài toán phức tạp hơn, điển hình là 2-SAT.
- Nhận biết được bài toán tìm tổ tiên trên cây và các phương pháp tiếp cận hiệu quả.

Đồ thị Liên thông mạnh và Thành phần Liên thông mạnh

Một đồ thị có hướng được gọi là liên thông mạnh (strongly connected) nếu tồn tại một đường đi từ một đỉnh bất kỳ đến tất cả các đỉnh khác trong đồ thị. Điều này ngụ ý rằng với hai đỉnh u và v bất kỳ, luôn có đường đi từ u đến v và cả đường đi từ v về u . Đây là một yêu cầu chặt chẽ hơn nhiều so với tính liên thông trên đồ thị vô hướng. Ví dụ, một chu trình đơn có hướng là một đồ thị liên thông mạnh, nhưng nếu một cạnh bị loại bỏ, tính chất này có thể bị phá vỡ ngay lập tức.

Trong thực tế, không phải toàn bộ đồ thị đều liên thông mạnh. Thay vào đó, chúng ta thường quan tâm đến các thành phần liên thông mạnh (Strongly Connected Components - SCCs). Một thành phần liên thông mạnh của đồ thị có hướng là một tập hợp con các đỉnh sao cho với mọi cặp đỉnh u, v trong tập hợp, đều có đường đi từ u đến v và từ v đến u . Quan trọng hơn, mỗi SCC là một tập hợp cực đại, nghĩa là không thể thêm bất kỳ đỉnh nào khác vào mà vẫn duy trì tính liên thông mạnh. Các thành phần liên thông mạnh tạo thành một phân hoạch của tập hợp các đỉnh của đồ thị, nghĩa là mỗi đỉnh thuộc về đúng một SCC.

Việc phân rã đồ thị thành các SCC giúp làm sáng tỏ cấu trúc sâu sắc của nó. Ví dụ, xét một đồ thị có các đỉnh 1, 2, 3, 4, 5, 6, 7. Sau khi phân tích, ta có thể tìm thấy các

SCC là $A = 1, 2$, $B = 3, 6, 7$, $C = 4$ và $D = 5$. Mỗi tập hợp này là một "khối" mà bên trong nó, mọi đỉnh đều có thể "giao tiếp" hai chiều với nhau. Các đỉnh 4 và 5 tự tạo thành các SCC riêng lẻ vì không có chu trình nào chứa chúng. Việc phân tích này biến một đồ thị có thể có chu trình phức tạp thành một tập hợp các khối liên thông mạnh, mở đường cho các phân tích ở cấp độ cao hơn.

Đồ thị Thành phần (Component Graph)

Sau khi xác định được tất cả các thành phần liên thông mạnh của một đồ thị gốc G , chúng ta có thể xây dựng một cấu trúc mới gọi là đồ thị thành phần (component graph). Trong đồ thị này, mỗi đỉnh đại diện cho một thành phần liên thông mạnh của G . Một cạnh có hướng được vẽ từ đỉnh đại diện cho SCC $C1$ đến đỉnh đại diện cho SCC $C2$ nếu và chỉ nếu tồn tại ít nhất một cạnh trong đồ thị gốc G đi từ một đỉnh thuộc $C1$ đến một đỉnh thuộc $C2$.

Một trong những thuộc tính quan trọng và hữu ích nhất của đồ thị thành phần là nó luôn là một đồ thị có hướng không có chu trình (Directed Acyclic Graph - DAG). Điều này là do nếu tồn tại một chu trình trong đồ thị thành phần, ví dụ từ $C1$ đến $C2$ và ngược lại, điều đó có nghĩa là có đường đi từ một đỉnh trong $C1$ đến một đỉnh trong $C2$ và ngược lại trong đồ thị gốc. Theo định nghĩa, tất cả các đỉnh trong $C1$ và $C2$ sẽ cùng nhau tạo thành một thành phần liên thông mạnh lớn hơn, điều này mâu thuẫn với giả định rằng $C1$ và $C2$ là các thành phần cực đại. Do đó, đồ thị thành phần phải không có chu trình.

Việc biến đổi đồ thị ban đầu thành một DAG mang lại lợi ích to lớn. Các đồ thị DAG dễ xử lý hơn nhiều so với các đồ thị có chu trình. Chúng ta có thể áp dụng các kỹ thuật mạnh mẽ như sắp xếp topo (topological sort) để xử lý các thành phần theo một thứ tự hợp lý, hoặc sử dụng quy hoạch động trên DAG để giải quyết các bài toán tối ưu. Ví dụ, với các SCC A, B, C, D đã nêu, đồ thị thành phần có thể có các cạnh như $B \rightarrow C$ và $B \rightarrow D$, cho thấy một luồng thông tin hoặc sự phụ thuộc từ thành phần B sang C và D . Cấu trúc DAG này là chìa khóa để giải quyết nhiều bài toán phức tạp, bao gồm cả 2-SAT.

Thuật toán Kosaraju và Ứng dụng trong bài toán 2-SAT

Thuật toán Kosaraju là một phương pháp kinh điển và hiệu quả để tìm tất cả các thành phần liên thông mạnh trong một đồ thị có hướng. Thuật toán này hoạt động trong thời gian tuyến tính $O(V+E)$ (với V là số đỉnh và E là số cạnh) và dựa trên việc thực hiện hai lượt duyệt theo chiều sâu (Depth-First Search - DFS) một cách thông minh.

Lượt duyệt đầu tiên của thuật toán Kosaraju được thực hiện trên đồ thị gốc G . Mục tiêu của lượt duyệt này là xây dựng một danh sách các đỉnh theo thứ tự thời gian kết thúc của chúng trong cây DFS. Cụ thể, ta duyệt qua tất cả các đỉnh. Nếu một đỉnh chưa được thăm, ta bắt đầu một thủ tục DFS từ đỉnh đó. Một đỉnh chỉ được thêm vào cuối danh sách sau khi tất cả các đỉnh con của nó trong cây DFS đã được duyệt xong.

(thứ tự post-order). Thứ tự này có một tính chất đặc biệt: các đỉnh thuộc các SCC "đích" (sink SCCs) trong đồ thị thành phần sẽ xuất hiện gần cuối danh sách.

Lượt duyệt thứ hai được thực hiện trên đồ thị chuyển vị GT, là đồ thị G với tất cả các cạnh được đảo ngược chiều. Ta duyệt qua các đỉnh theo thứ tự ngược lại với thứ tự chúng được thêm vào danh sách ở bước một. Đối với mỗi đỉnh u chưa được gán vào SCC nào, ta bắt đầu một DFS từ u trên đồ thị GT. Tất cả các đỉnh có thể được thăm từ u trong lượt DFS này sẽ cùng nhau tạo thành một thành phần liên thông mạnh. Bằng cách này, thuật toán sẽ lần lượt xác định từng SCC một cách chính xác.

Một ứng dụng tiêu biểu và mạnh mẽ của việc tìm SCC là giải quyết bài toán 2-Satisfiability (2-SAT). Bài toán này yêu cầu xác định xem có tồn tại một phép gán giá trị chân lý (đúng/sai) cho một tập hợp các biến boolean để một biểu thức mệnh đề cho trước được thỏa mãn hay không. Biểu thức này có dạng hội của nhiều mệnh đề con, trong đó mỗi mệnh đề con là phép tuyển của đúng hai "literal" (một biến hoặc phủ định của một biến), ví dụ $(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3)$. Để giải quyết, ta xây dựng một đồ thị bao hàm (implication graph). Mỗi mệnh đề $(a \vee b)$ tương đương với hai bao hàm $(\neg a \Rightarrow b)$ và $(\neg b \Rightarrow a)$. Mỗi literal (ví dụ x_i và $\neg x_i$) trở thành một đỉnh trong đồ thị, và mỗi bao hàm $p \Rightarrow q$ trở thành một cạnh có hướng từ đỉnh p đến đỉnh q .

Điều kiện cốt lõi để một biểu thức 2-SAT có lời giải là: không tồn tại biến x_i nào mà x_i và phủ định của nó $\neg x_i$ cùng nằm trong một thành phần liên thông mạnh. Nếu chúng cùng thuộc một SCC, điều đó có nghĩa là tồn tại đường đi từ x_i đến $\neg x_i$ và ngược lại. Điều này tạo ra một mâu thuẫn logic: x_i đúng kéo theo $\neg x_i$ đúng, và $\neg x_i$ đúng cũng kéo theo x_i đúng. Đây là một vòng lặp vô lý không thể thỏa mãn. Do đó, chỉ cần dùng thuật toán Kosaraju để tìm các SCC và kiểm tra điều kiện này là ta có thể xác định tính giải được của bài toán.

Nếu một lời giải tồn tại, ta có thể xây dựng nó bằng cách xử lý các thành phần trong đồ thị thành phần theo thứ tự sắp xếp topo ngược. Khi xét một thành phần, nếu các biến trong đó chưa được gán giá trị, ta sẽ gán giá trị cho chúng. Một chiến lược phổ biến là: nếu biến x_i và $\neg x_i$ nằm ở các thành phần khác nhau, ta sẽ gán giá trị false cho literal nào xuất hiện trong thành phần được xử lý sau theo thứ tự topo (tức là được xử lý trước trong thứ tự topo ngược). Ví dụ, nếu thành phần chứa $\neg x_1$ đứng trước thành phần chứa x_1 trong thứ tự topo ngược, ta sẽ xử lý thành phần của $\neg x_1$ trước và có thể gán x_1 giá trị true. Quá trình này đảm bảo không tạo ra mâu thuẫn và cung cấp một phép gán chân lý hợp lệ.

Tìm Tổ tiên thứ k bằng Binary Lifting

Chuyển sang các thuật toán trên cây, một bài toán truy vấn phổ biến là tìm tổ tiên thứ k của một đỉnh cho trước. Cụ thể, cho một cây có gốc, với một đỉnh u và một số nguyên k, ta cần tìm đỉnh nằm trên đường đi duy nhất từ u về gốc và cách u đúng k cạnh. Cách tiếp cận đơn giản nhất là đi ngược lên từng cha một k lần, nhưng phương pháp này có độ phức tạp $O(k)$ cho mỗi truy vấn, quá chậm cho các bài toán có nhiều truy vấn hoặc k lớn.

Để giải quyết vấn đề này một cách hiệu quả, người ta thường sử dụng kỹ thuật Binary Lifting (còn gọi là nhảy nhị phân hoặc bảng thừa trên cây). Ý tưởng cốt lõi là tiền tính toán và lưu trữ thông tin về các tổ tiên ở khoảng cách là lũy thừa của 2. Cụ thể, với mỗi đỉnh u, ta sẽ lưu trữ không chỉ cha trực tiếp của nó (2^0 -tổ tiên), mà còn cả tổ tiên thứ 2 (2^1), thứ 4 (2^2), thứ 8 (2^3), và cứ thế cho đến khi vượt ra ngoài cây. Mảng `up[u][i]` sẽ lưu trữ tổ tiên thứ 2^i của đỉnh u.

Với các thông tin đã được tiền tính toán này, việc tìm tổ tiên thứ k có thể được thực hiện trong thời gian $O(\log k)$. Ta phân tích k thành tổng các lũy thừa của 2 (biểu diễn nhị phân của k). Ví dụ, để tìm tổ tiên thứ 13, ta biết rằng $13 = 8 + 4 + 1$. Do đó, ta có thể nhảy từ u lên tổ tiên thứ 8, sau đó từ đỉnh kết quả nhảy tiếp lên tổ tiên thứ 4, và cuối cùng nhảy lên tổ tiên thứ 1. Mỗi bước nhảy này chỉ mất một thao tác tra cứu trong bảng đã tiền tính toán. Tuy nhiên, các tài liệu được cung cấp cho chương này tập trung chủ yếu vào phân rã đồ thị và không đi sâu vào chi tiết cài đặt hay phân tích của kỹ thuật Binary Lifting. Do đó, phần này chỉ mang tính giới thiệu về bài toán và tên của phương pháp giải quyết hiệu quả tiêu chuẩn, khuyến khích người đọc tìm hiểu thêm từ các nguồn chuyên sâu về thuật toán trên cây.

Điểm cốt lõi

- > Một đồ thị có hướng là liên thông mạnh nếu tồn tại đường đi từ mọi đỉnh đến mọi đỉnh khác. Các thành phần liên thông mạnh (SCC) là các phân hoạch cực đại của đồ thị vào các tập đỉnh như vậy.
- > Thuật toán Kosaraju sử dụng hai lượt duyệt DFS—một trên đồ thị gốc và một trên đồ thị chuyển vị—để tìm tất cả các SCC trong thời gian tuyến tính $O(V+E)$.
- > Đồ thị thành phần, nơi mỗi đỉnh đại diện cho một SCC, luôn là một đồ thị có hướng không có chu trình (DAG), cho phép áp dụng các thuật toán như sắp xếp topo và quy hoạch động.
- > Bài toán 2-SAT có thể giải được khi và chỉ khi không có biến x và phủ định $\neg x$ của nó cùng thuộc một thành phần liên thông mạnh trong đồ thị bao hàm tương ứng.
- > Kỹ thuật Binary Lifting cho phép trả lời truy vấn tìm tổ tiên thứ k trên cây trong thời gian $O(\log k)$ sau một bước tiền xử lý.

Câu hỏi ôn tập

1. Tại sao đồ thị thành phần (component graph) của một đồ thị có hướng bất kỳ lại luôn là một đồ thị có hướng không chu trình (DAG)? Điều gì sẽ xảy ra nếu nó có chu trình?
2. Trong thuật toán Kosaraju, tại sao lượt duyệt DFS thứ hai phải được thực hiện trên đồ thị chuyển vị (GT) và theo thứ tự ngược lại của thời gian kết thúc từ lượt duyệt đầu tiên? Vai trò của mỗi yếu tố này là gì?
3. Mô tả cách xây dựng đồ thị bao hàm (implication graph) từ một biểu thức 2-SAT. Một cạnh từ p đến q trong đồ thị này có ý nghĩa logic là gì?
4. So sánh ưu và nhược điểm của việc tìm tổ tiên thứ k bằng cách duyệt ngược từng cha so với kỹ thuật Binary Lifting về mặt thời gian truy vấn và chi phí bộ nhớ/tiền xử lý.

Chương 8: Các Chủ đề Nâng cao: Số học, Hình học và Xác suất

Khi đã nắm vững các cấu trúc dữ liệu và thuật toán cơ bản, hành trình chinh phục lập trình thi đấu sẽ đưa bạn đến với những lĩnh vực toán học chuyên sâu hơn. Ba trong số những trụ cột quan trọng nhất là lý thuyết số, hình học tính toán và xác suất. Việc thành thạo các công cụ trong những lĩnh vực này không chỉ giúp giải quyết các bài toán hóc búa mà còn mở ra một tư duy thuật toán hoàn toàn mới. Chương cuối cùng này sẽ trang bị cho bạn những khái niệm và kỹ thuật nền tảng nhưng đầy sức mạnh trong từng lĩnh vực, giúp bạn tự tin đối mặt với các thử thách đa dạng và phức tạp nhất trong các kỳ thi.

Mục tiêu học tập

- Áp dụng được thuật toán Euclid mở rộng để giải phương trình Diophantine và hiểu nguyên lý của Định lý số dư Trung Hoa.
- Phân biệt và tính toán các đại lượng xác suất cơ bản như giá trị kỳ vọng, phân phối nhị thức, và ứng dụng phương pháp Las Vegas trong giải quyết bài toán.
- Sử dụng lớp complex trong C++ để giải quyết các bài toán hình học và áp dụng công thức tính diện tích đa giác.
- Mô tả được các bước của thuật toán Andrew để tìm bao lồi của một tập hợp điểm.

Lý thuyết Số Ứng dụng

Trong lập trình thi đấu, nhiều bài toán yêu cầu xử lý các con số cực lớn, vượt xa khả năng lưu trữ của các kiểu dữ liệu 64-bit như long long. Đây là lúc số học mô-đun (modular arithmetic) phát huy tác dụng. Kỹ thuật này cho phép chúng ta thực hiện các phép toán trên các số nguyên trong một vành hữu hạn, thường là tìm kết quả của một biểu thức lớn "modulo m", tức là phần dư của nó khi chia cho m. Một giá trị m thường gặp là $10^9 + 7$. Phép toán $x \bmod m$ trả về phần dư khi x được chia cho m, ví dụ $17 \bmod 5 = 2$. Việc áp dụng số học mô-đun giúp giữ cho các kết quả trung gian luôn nằm trong giới hạn cho phép, tránh bị tràn số mà vẫn đảm bảo tính đúng đắn của kết quả cuối cùng.

Một công cụ nền tảng khác trong lý thuyết số là thuật toán Euclid dùng để tìm ước chung lớn nhất (greatest common divisor - GCD) của hai số nguyên. Tuy nhiên, phiên bản mở rộng của nó, thuật toán Euclid mở rộng, còn làm được nhiều hơn thế. Nó không chỉ tìm ra $\gcd(a, b)$ mà còn tìm được một cặp số nguyên (x, y) thỏa mãn phương trình $ax + by = \gcd(a, b)$. Nguyên lý của thuật toán này dựa trên việc quay lui theo các bước của thuật toán Euclid thông thường để biểu diễn gcd dưới dạng một tổ hợp tuyến tính của a và b.

Sức mạnh của thuật toán Euclid mở rộng được thể hiện rõ nhất khi giải phương trình Diophantine tuyến tính có dạng $ax + by = c$, trong đó a, b, c là các hằng số nguyên và ta cần tìm nghiệm nguyên (x, y) . Một phương trình như vậy chỉ có nghiệm nguyên

khi và chỉ khi c chia hết cho $\gcd(a, b)$. Nếu điều kiện này được thỏa mãn, ta có thể dùng thuật toán Euclid mở rộng để tìm một cặp (x', y') sao cho $ax' + by' = \text{extgcd}(a, b)$. Sau đó, một nghiệm (x_0, y_0) của phương trình ban đầu có thể được tìm thấy bằng cách nhân cả hai vế với $c / \text{extgcd}(a, b)$. Ví dụ, để giải $39x + 15y = 12$, ta có $\gcd(39, 15) = 3$ và 12 chia hết cho 3. Thuật toán Euclid mở rộng cho ta $39 \text{ imes } 2 + 15 \text{ imes } (-5) = 3$. Nhân cả hai vế với 4, ta được $39 \text{ imes } 8 + 15 \text{ imes } (-20) = 12$, vậy một nghiệm là $(x=8, y=-20)$.

Khi đã có một nghiệm (x_0, y_0) , ta có thể tìm ra vô số nghiệm khác. Tất cả các nghiệm của phương trình Diophantine sẽ có dạng $(x_0 + k \text{ racbd}, y_0 - k \text{ racad})$ với $d = \gcd(a, b)$ và k là một số nguyên bất kỳ. Ngoài ra, Định lý số dư Trung Hoa (Chinese Remainder Theorem) cung cấp một phương pháp mạnh mẽ để giải quyết một hệ các phương trình đồng dư, ví dụ như tìm một số x thỏa mãn đồng thời $x \equiv a_1 \pmod{m_1}$, $x \equiv a_2 \pmod{m_2}, \dots$, với điều kiện các mô-đun $m_1, m_2, \dots$ phải đôi một nguyên tố cùng nhau. Định lý này có ứng dụng quan trọng trong các bài toán yêu cầu kết hợp các kết quả từ nhiều phép toán mô-đun khác nhau.

Xác suất và Thuật toán Ngẫu nhiên

Một trong những khái niệm trung tâm của xác suất là giá trị kỳ vọng (expected value), đại diện cho giá trị trung bình của một biến ngẫu nhiên sau vô số lần thử. Một tính chất cực kỳ hữu ích của nó là tuyến tính của kỳ vọng, phát biểu rằng kỳ vọng của tổng các biến ngẫu nhiên bằng tổng các kỳ vọng của chúng, bất kể chúng có phụ thuộc vào nhau hay không. Ví dụ, xét bài toán ném n quả bóng vào n chiếc hộp một cách ngẫu nhiên. Để tính số hộp rỗng kỳ vọng, thay vì xét tất cả các trường hợp phức tạp, ta có thể tính xác suất để một hộp cụ thể bị rỗng là $((n-1)/n)^n$. Do có n hộp, theo tính tuyến tính của kỳ vọng, số hộp rỗng kỳ vọng đơn giản là $n \text{ imes } ((n-1)/n)^n$. Cách tiếp cận này biến một bài toán tổ hợp có vẻ đáng sợ thành một phép tính đơn giản.

Trong lập trình thi đấu, việc nhận biết được phân phối xác suất của một biến ngẫu nhiên là rất quan trọng. Phân phối đều (uniform distribution) xảy ra khi mọi kết quả có thể có đều có cùng một xác suất, ví dụ như tung một con xúc xắc cân đối. Phân phối nhị thức (binomial distribution) mô tả số lần thành công trong n phép thử độc lập, mỗi phép thử có xác suất thành công là p . Công thức tính xác suất để có đúng x lần thành công là $P(X=x) = \binom{n}{x} p^x (1-p)^{n-x}$, trong đó $\binom{n}{x}$ là số cách chọn x vị trí thành công từ n phép thử. Ví dụ, xác suất tung được mặt sáu đúng 3 lần trong 10 lần tung xúc xắc là $\binom{10}{3} (1/6)^3 (5/6)^7$. Giá trị kỳ vọng của phân phối nhị thức là np , một công thức rất trực quan.

Lý thuyết xác suất không chỉ dùng để phân tích mà còn là nền tảng cho một lớp thuật toán mạnh mẽ gọi là thuật toán ngẫu nhiên. Một ví dụ điển hình là thuật toán Las Vegas. Khác với thuật toán Monte Carlo (có thể cho kết quả sai), thuật toán Las Vegas luôn đảm bảo đưa ra câu trả lời chính xác, nhưng thời gian chạy của nó là một biến ngẫu nhiên. Ý tưởng cốt lõi là nếu một giải pháp ngẫu nhiên có xác suất cao là một lời giải hợp lệ, ta có thể liên tục tạo ra các giải pháp ngẫu nhiên cho đến khi tìm

được một lời giải đúng.

Xét bài toán tô màu các đỉnh của một đồ thị có m cạnh bằng hai màu sao cho ít nhất $m/2$ cạnh có hai đầu được tô khác màu. Thay vì tìm kiếm một cách có hệ thống, ta có thể thử một thuật toán Las Vegas: tô màu ngẫu nhiên cho mỗi đỉnh (50% cho mỗi màu). Với một cạnh bất kỳ, xác suất hai đầu của nó có màu khác nhau là $1/2$. Do đó, số cạnh kỳ vọng có hai đầu khác màu là $m/2$. Vì giá trị kỳ vọng chính bằng ngưỡng yêu cầu, một phép tô màu ngẫu nhiên có khả năng cao sẽ là một lời giải hợp lệ. Trong thực tế, phương pháp này thường tìm ra lời giải rất nhanh, thể hiện sự hiệu quả của việc đưa yếu tố ngẫu nhiên vào thiết kế thuật toán.

Các Kỹ thuật trong Hình học Tính toán

Hình học tính toán là một lĩnh vực đầy thách thức vì nó đòi hỏi sự chính xác tuyệt đối và phải xử lý được nhiều trường hợp đặc biệt (edge cases). Một ví dụ kinh điển là bài toán tính diện tích của một đa giác. Một cách tiếp cận trực quan là chia đa giác thành các tam giác, tính diện tích từng tam giác rồi cộng lại. Tuy nhiên, phương pháp này tiềm ẩn một cạm bẫy: làm thế nào để chia đa giác một cách chính xác? Với một tứ giác không lồi, việc chọn hai đỉnh đối diện bất kỳ để vẽ đường chéo có thể tạo ra một đoạn thẳng nằm ngoài đa giác. Con người có thể dễ dàng nhận ra đâu là cách chia đúng, nhưng việc lập trình để máy tính tự động đưa ra lựa chọn này lại rất phức tạp và dễ phát sinh lỗi.

May mắn thay, toán học cung cấp một giải pháp thanh lịch và mạnh mẽ hơn nhiều. Đối với một tứ giác có các đỉnh (x_1, y_1) , (x_2, y_2) , (x_3, y_3) , và (x_4, y_4) theo thứ tự, diện tích (có dấu) của nó có thể được tính bằng công thức $1/2 * (x_1y_2 - x_2y_1 + x_2y_3 - x_3y_2 + x_3y_4 - x_4y_3 + x_4y_1 - x_1y_4)$. Công thức này, một trường hợp đặc biệt của công thức Shoelace, có ưu điểm vượt trội: nó cực kỳ dễ cài đặt, không cần xử lý các trường hợp đặc biệt cho đa giác lồi hay lõm, và có thể tổng quát hóa cho bất kỳ đa giác đơn nào. Đây là một minh chứng cho thấy việc áp dụng đúng công thức toán học có thể đơn giản hóa đáng kể một vấn đề lập trình phức tạp.

Để làm việc với các bài toán hình học một cách hiệu quả hơn trong C++, thư viện chuẩn cung cấp lớp complex. Một số phức có dạng $x + yi$ có thể được diễn giải một cách tự nhiên trong hình học như một điểm có tọa độ (x, y) trên mặt phẳng hai chiều, hoặc một vector đi từ gốc tọa độ đến điểm đó. Việc sử dụng complex để biểu diễn điểm và vector giúp trừu tượng hóa các phép toán hình học. Các thao tác như cộng, trừ vector, xoay một điểm quanh gốc tọa độ, hay tính độ dài vector đều trở nên ngắn gọn và dễ đọc hơn rất nhiều so với việc xử lý riêng lẻ từng tọa độ x và y . Đây là một công cụ không thể thiếu cho bất kỳ lập trình viên thi đấu nào muốn giải quyết các bài toán hình học một cách nhanh chóng và chính xác.

Thuật toán Andrew tìm Bao lồi

Bài toán bao lồi (convex hull) là một trong những bài toán cơ bản và quan trọng nhất của hình học tính toán. Mục tiêu là tìm ra đa giác lồi nhỏ nhất chứa tất cả các điểm trong một tập hợp cho trước. Hãy tưởng tượng việc đóng một sợi dây chun quanh một cụm đinh trên bảng; hình dạng mà sợi dây chun tạo ra chính là bao lồi. Thuật toán Andrew là một phương pháp hiệu quả để giải quyết bài toán này, với độ phức tạp thời gian là $O(n \log n)$.

Ý tưởng chính của thuật toán Andrew là chia bài toán thành hai phần đơn giản hơn: xây dựng bao trên (upper hull) và bao dưới (lower hull) của tập hợp điểm. Để làm được điều này, bước đầu tiên và quan trọng nhất là sắp xếp tất cả các điểm theo thứ tự từ điển, tức là ưu tiên theo tọa độ x, sau đó đến tọa độ y. Sau khi sắp xếp, thuật toán sẽ duyệt qua các điểm để xây dựng hai nửa của bao lồi một cách độc lập.

Để xây dựng bao dưới, ta duyệt qua các điểm đã sắp xếp từ trái sang phải. Ta duy trì một danh sách các điểm ứng cử viên cho bao dưới. Với mỗi điểm mới, ta xét bộ ba điểm gồm hai điểm cuối cùng trong danh sách ứng cử viên và điểm mới. Ta cần kiểm tra xem việc thêm điểm mới có tạo ra một "cú rẽ trái" hay không. Nếu bộ ba điểm tạo thành một "cú rẽ phải" hoặc thẳng hàng, điều đó có nghĩa là điểm ở giữa (điểm cuối cùng trong danh sách) nằm bên trong hoặc trên cạnh của bao lồi mới, do đó nó không còn là một đỉnh của bao lồi nữa và cần bị loại bỏ. Ta lặp lại quá trình loại bỏ này cho đến khi bộ ba điểm tạo thành một cú rẽ trái, sau đó mới thêm điểm mới vào danh sách. Quá trình này đảm bảo rằng danh sách ứng cử viên luôn duy trì tính chất lồi.

Việc xây dựng bao trên được thực hiện theo một quy trình tương tự. Ta duyệt qua các điểm đã sắp xếp theo thứ tự ngược lại, từ phải sang trái, và cũng duy trì một danh sách ứng cử viên cho bao trên. Lần này, ta cũng kiểm tra hướng rẽ của bộ ba điểm và loại bỏ các điểm ở giữa nếu chúng không tạo thành một "cú rẽ trái" (khi nhìn từ phải sang trái). Cuối cùng, bao lồi hoàn chỉnh được tạo thành bằng cách kết hợp danh sách điểm của bao dưới và bao trên, loại bỏ các điểm bị trùng lặp ở hai đầu. Độ phức tạp của toàn bộ thuật toán bị chi phối bởi bước sắp xếp ban đầu, do đó là $O(n \log n)$.

Điểm cốt lõi

- > Phương trình Diophantine $ax + by = c$ có nghiệm nguyên khi và chỉ khi c chia hết cho $\gcd(a, b)$, và có thể giải hiệu quả bằng thuật toán Euclid mở rộng.
- > Tính chất tuyến tính của kỳ vọng cho phép tính kỳ vọng của một tổng bằng tổng các kỳ vọng, là một công cụ mạnh mẽ để đơn giản hóa nhiều bài toán xác suất phức tạp.
- > Thuật toán Las Vegas là một dạng thuật toán ngẫu nhiên luôn cho kết quả đúng, với thời gian chạy không xác định nhưng thường nhanh trong thực tế, hữu ích khi một lời giải ngẫu nhiên có xác suất thành công cao.
- > Lớp complex trong C++ là một công cụ mạnh mẽ để biểu diễn điểm và vector, giúp mã nguồn cho các bài toán hình học tính toán trở nên ngắn gọn và dễ hiểu hơn.

> Thuật toán Andrew tìm bao lồi bằng cách sắp xếp các điểm và xây dựng bao trên và bao dưới một cách độc lập, dựa trên việc kiểm tra hướng "rẽ" của các bộ ba điểm liên tiếp để duy trì tính lồi.

Câu hỏi ôn tập

1. Làm thế nào bạn có thể tìm một nghiệm nguyên (x, y) cho phương trình $27x + 18y = 9$? Điều gì sẽ xảy ra nếu vế phải là 10 thay vì 9?
2. Một giải đấu có n đội, mỗi đội đấu với mọi đội khác đúng một lần. Giả sử trong mỗi trận đấu, mỗi đội có 50% cơ hội thắng. Giá trị kỳ vọng của tổng số trận thắng của một đội bất kỳ là bao nhiêu?
3. Tại sao việc sử dụng công thức diện tích đa giác tổng quát (công thức Shoelace) lại thường ưu việt hơn phương pháp chia đa giác thành các tam giác trong lập trình thi đấu?
4. Trong thuật toán Andrew, tại sao cần phải sắp xếp các điểm trước khi xây dựng bao lồi? Việc kiểm tra "hướng rẽ" đóng vai trò gì trong việc loại bỏ các điểm không thuộc bao lồi?